Studio Nx

P1-28 계정 찾기 JS — 함수 설명 합본 (JS 165~176)

2026-10-15

목차

1. ai-notes/javascript/P1-28 - 비밀번호 찾기/165_recover_showView.md
2. ai-notes/javascript/P1-28 - 비밀번호 찾기/166_recover_selectTab.md
3. ai-notes/javascript/P1-28 - 비밀번호 찾기/167_recover_initRecoverTabs.md
4. ai-notes/javascript/P1-28 - 비밀번호 찾기/168_recover_initRecover.md
5. ai-notes/javascript/P1-28 - 비밀번호 찾기/169_recover_initRecoverFromHash.md
6. ai-notes/javascript/P1-28 - 비밀번호 찾기/170_recover_showSentView.md
7. ai-notes/javascript/P1-28 - 비밀번호 찾기/171_recover_initRecoverForms.md
8. ai-notes/javascript/P1-28 - 비밀번호 찾기/172_recover_resendCooldown.md
9. ai-notes/javascript/P1-28 - 비밀번호 찾기/173_recover_initRecoverResend.md
10. ai-notes/javascript/P1-28 - 비밀번호 찾기/174_reset_readResetToken.md
11. ai-notes/javascript/P1-28 - 비밀번호 찾기/175_reset_initResetPasswordRules.md
12. ai-notes/javascript/P1-28 - 비밀번호 찾기/176_reset_initResetForm.md

JS 165 — `showView()` (모달 안에서 로그인 화면과 복구 화면 바꾸기)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (새 파일)
작업	P1-28 계정 찾기 화면: JS 첫 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	개요 70 (HTML과 CSS), 백엔드 164
다음	탭 전환과 키보드 이동, 그다음 열기 연결(`Can't sign in?`, `#recover`)

1. 하는 일

로그인 모달(`#login-modal`) 안의 두 화면, `data-view="login"`과 `data-view="recover"` 중 하나만 보이게 바꿉니다.

하는 일	설명
화면 전환	`name`의 `[data-view]`만 보이고 나머지는 숨깁니다(`hidden` 속성)
제목 변경	`#login-modal-title`의 글자를 `data-title-<name>` 속성의 값으로 바꿉니다(`Sign In` ⇄ `Can't sign in?`). 속성이 없으면 그대로입니다
커서 이동	새 화면에서 **보이는 첫 입력칸**으로 커서를 옮깁니다
반환	바꿨으면 `true`, 모르는 이름이거나 모달, 화면이 없으면 아무것도 바꾸지 않고 `false`

2. 코드

```
export const RECOVER_VIEWS = ['login', 'recover'];

export function showView(modal, name) {
  if (!modal || RECOVER_VIEWS.indexOf(name) < 0) return false;
  const views = modal.querySelectorAll('[data-view]');
  if (views.length === 0) return false;

  let shown = null;
  for (const view of views) {
    const on = view.getAttribute('data-view') === name;
    view.hidden = !on;
    if (on) shown = view;
  }
  if (shown === null) return false;

  const title = modal.querySelector('#login-modal-title');
  const text = title ? title.getAttribute('data-title-' + name) : null;
  if (title && text !== null) title.textContent = text;

  const inputs = shown.querySelectorAll('input:not([type="hidden"]):not([disabled])');
  for (const input of inputs) {
    if (input.closest('[hidden]') === null) {
```

```
      input.focus();
      break;
    }
  }
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const showView = (modal, name) => {
  if (!modal || !RECOVER_VIEWS.includes(name)) return false;
  const views = [...modal.querySelectorAll('[data-view]')];
  const shown = views.find((v) => v.dataset.view === name);
  if (!shown) return false;
  views.forEach((v) => { v.hidden = v !== shown; });

  const title = modal.querySelector('#login-modal-title');
  const text = title?.dataset[`title${name[0].toUpperCase()}${name.slice(1)}`];
  if (text !== undefined) title.textContent = text;

  [...shown.querySelectorAll('input:not([type="hidden"]):not([disabled])')]
    .find((input) => !input.closest('[hidden]'))?.focus();
  return true;
};
```

최신 문법	실제 코드	뜻
`RECOVER_VIEWS.includes(name)`	`indexOf(name) < 0`	목록에 있는지 확인합니다
`views.find(…)`	`for` 반복에서 `shown`에 담기	이름이 같은 화면을 찾습니다
`title?.dataset[…]`	`title ? title.getAttribute(…) : null`	값이 없으면 건너뜁니다(옵셔널 체이닝)
`[...].find(…)?.focus()`	`for`와 `break`	조건에 맞는 첫 칸에만 커서를 줍니다

문법 풀이

식	뜻
`view.hidden = !on`	`hidden` 속성을 켜고 끕니다. CSS에서 `display`를 따로 준 요소는 이 속성이 안 먹을 수 있어서 `[hidden] { display: none }` 규칙이 필요합니다(탭에 이미 더했습니다)
`input.closest('[hidden]')`	입력칸의 조상 중 `hidden`인 것이 있는지 찾습니다. 없으면 `null`이라 "지금 보이는 칸"입니다
`input:not([type="hidden"]):not([disabled])`	숨김 입력과 비활성 입력은 커서를 받을 수 없어서 뺍니다

3. 왜 이렇게

선택	이유
화면 전환을 한 함수에 모음	열기, 닫기, 뒤로 가기, 보낸 뒤 모두가 같은 전환을 씁니다. 각자 `hidden`을 만지면 어긋납니다
커서를 옮김	키보드와 화면 낭독기 사용자는 화면이 바뀌어도 커서가 전에 누른 버튼에 남아 있으면 위치를 잃습니다. 새 화면의 첫 입력칸으로 옮기면 바로 입력할 수 있습니다
보이는 칸만 선택 (`closest('[hidden]')`)	복구 화면에는 Email 패널의 입력칸도 있는데, 숨은 패널의 칸에 커서를 주면 아무 일도 일어나지 않거나 엉뚱한 곳으로 갑니다. 시험으로 확인했습니다

선택	이유
제목을 데이터 속성에서 읽음	화면 글자를 JS에 적지 않고 HTML에 둡니다. 나중에 글자를 바꿔도 JS는 그대로입니다
모르는 이름은 `false`로 아무것도 바꾸지 않음	오타(`'RECOVER'`)로 두 화면이 모두 숨는 일을 막습니다. 예외를 던지지 않고 값으로 알려서, 이벤트 처리기가 끊기지 않게 합니다
이미 그 화면이면 다시 불러도 같은 결과	두 번 눌러도 안전합니다
탭, 폼 상태는 건드리지 않음	이 함수는 화면 전환 한 가지만 합니다. 탭 선택과 입력값 비우기는 다음 함수들이 맡습니다

4. 시험 (실제 서버, 헤드리스 Chrome, 27개 통과)

`home.html`과 `join.html` 두 페이지에서 같은 시험을 했습니다.

확인	결과
시작 상태: 로그인 화면이 보이고 제목이 `Sign In`	✓ ✓
`recover`로 바꿈: 복구 화면만 **실제로** 보임(`getComputedStyle`), 제목이 `Can't sign in?`, 커서가 Password 탭의 이메일 칸	✓ ✓
이미 그 화면이면 다시 불러도 같은 결과	✓ ✓
Email 패널이 보이는 상태면 그 패널의 입력칸에 커서(숨은 Password 패널의 칸은 건너뜀)	✓ ✓
`login`으로 돌아옴: 제목 `Sign In`, 커서가 로그인 이메일 칸	✓ ✓
모르는 이름 6종(`'x'`, `''`, `undefined`, `null`, `'RECOVER'`, `1`)은 `false`이고 아무것도 바뀌지 않음	✓ (12개)
모달이 `null`이면 `false`(오류 없음)	✓ ✓
제목 속성이 없으면 제목은 그대로이고 화면은 바뀜	✓
`[data-view]`가 하나도 없는 모달은 `false`이고 제목도 그대로	✓
내보내는 상수	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`hidden`을 반대로	8개 실패
제목 바꾸기 제거	2개 실패
커서 이동 제거	8개 실패
숨은 패널의 칸도 선택	2개 실패
이름 검사 제거	3개 실패
제목 속성이 없어도 덮어씀	1개 실패
`[data-view]`가 없을 때의 조기 반환 제거	**실패 없음**

마지막은 시험의 빈틈이 아니라 중복 방어입니다. 그 줄을 지워도 바로 아래의 `shown === null` 검사가 같은 경우를 `false`로 막기 때문입니다. 앞의 검사는 쓸모없는 반복을 피하는 빠른 반환입니다.

5. 아직 이 함수를 부르는 곳이 없습니다

화면에서 실제로 쓰이려면 다음 함수들이 필요합니다. 지금은 `Can't sign in?`을 눌러도 아무 일도 일어나지 않습니다.

- 탭 전환과 키보드 이동
- `Can't sign in?` 버튼과 `home.html#recover`로 복구 화면을 여는 연결

JS 166 — `selectTab()` (복구 화면의 탭 고르기)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js`
작업	P1-28 계정 찾기 화면: JS 두 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	165 `showView()`
다음	탭 클릭과 화살표 키 이동(이 함수를 부르는 처리기), 그다음 열기 연결

1. 하는 일

복구 화면의 두 탭(`password`, `email`) 중 하나를 고릅니다.

하는 일	설명
탭 표시	고른 탭만 `aria-selected="true"`와 `tabindex="0"`, 나머지는 `"false"`와 `"-1"`
패널 전환	짝이 되는 패널(`recover-panel-<이름>`)만 보이고 나머지는 숨김
값 보존	입력칸의 값과 오류 문구는 **건드리지 않음**: 탭을 오가도 적던 이메일이 남습니다
커서	`focus`가 `true`이거나, 숨겨질 패널의 칸에 커서가 있었으면 고른 탭으로 옮김
반환	바꿨으면 `true`, 모르는 이름, 모달 없음, 탭이나 패널이 없으면 아무것도 바꾸지 않고 `false`

2. 코드

```
export const RECOVER_TABS = ['password', 'email'];

export function selectTab(modal, name, focus) {
  if (!modal || RECOVER_TABS.indexOf(name) < 0) return false;
  const tabs = modal.querySelectorAll('[role="tab"]');
  const panels = modal.querySelectorAll('[role="tabpanel"]');
  const tabId = 'recover-tab-' + name;
  const panelId = 'recover-panel-' + name;

  let chosen = null;
  let chosenPanel = null;
  for (const tab of tabs) if (tab.id === tabId) chosen = tab;
  for (const panel of panels) if (panel.id === panelId) chosenPanel = panel;
  if (chosen === null || chosenPanel === null) return false;

  for (const tab of tabs) {
    const on = tab === chosen;
    tab.setAttribute('aria-selected', on ? 'true' : 'false');
    tab.setAttribute('tabindex', on ? '0' : '-1');
  }
  const active = document.activeElement;
  let lostFocus = false;
  for (const panel of panels) {
    if (panel !== chosenPanel && active && panel.contains(active)) lostFocus = true;
```

```
    panel.hidden = panel !== chosenPanel;
  }
  if (focus === true || lostFocus) chosen.focus();
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const selectTab = (modal, name, focus = false) => {
  if (!modal || !RECOVER_TABS.includes(name)) return false;
  const tabs = [...modal.querySelectorAll('[role="tab"]')];
  const panels = [...modal.querySelectorAll('[role="tabpanel"]')];
  const chosen = tabs.find((t) => t.id === `recover-tab-${name}`);
  const chosenPanel = panels.find((p) => p.id === `recover-panel-${name}`);
  if (!chosen || !chosenPanel) return false;

  tabs.forEach((t) => {
    t.setAttribute('aria-selected', String(t === chosen));
    t.tabIndex = t === chosen ? 0 : -1;
  });
  const lostFocus = panels.some((p) => p !== chosenPanel && p.contains(document.activeElement));
  panels.forEach((p) => { p.hidden = p !== chosenPanel; });
  if (focus === true || lostFocus) chosen.focus();
  return true;
};
```

최신 문법	실제 코드	뜻
`tabs.find(…)`	`for` 반복에서 변수에 담기	이름이 같은 탭을 찾습니다
`panels.some(…)`	`for` 안의 `if`로 `lostFocus` 표시	숨겨질 패널 안에 커서가 있는지 봅니다
`String(t === chosen)`	`on ? 'true' : 'false'`	참, 거짓을 글자로 바꿉니다
`t.tabIndex = …`	`setAttribute('tabindex', …)`	같은 일입니다

문법 풀이

식	뜻
`aria-selected`	화면 낭독기에 "이 탭이 선택됨"을 알립니다
`tabindex="0"` / `"-1"`	**Tab 키가 선택된 탭 하나에만 멈추게** 합니다. 나머지 탭은 화살표 키로 옮깁니다(WAI-ARIA 탭 패턴). 탭이 둘뿐이어도 이 패턴을 지켜서, 다음 단계의 화살표 키 이동과 맞물립니다
`panel.contains(active)`	지금 커서가 그 패널 안의 요소인지 확인합니다

3. 왜 이렇게

선택	이유
값과 오류 문구를 지우지 않음	사용자가 Password 탭에 이메일을 적다가 Email 탭을 보고 돌아와도 적은 것이 남습니다. 비우는 일은 모달을 닫을 때의 몫입니다
숨겨질 패널의 칸에 커서가 있었으면 탭으로 옮김	시험을 하다 발견했습니다(아래 §5). 숨은 칸에서는 브라우저가 커서를 빼앗아 `body`로 보내서, 키보드 사용자가 위치를 잃습니다
`focus === true`로 엄격하게 비교	`1`이나 `'false'` 같은 값이 실수로 커서를 옮기지 않게 합니다

선택	이유
`focus` 인자를 따로 둠	클릭은 브라우저가 이미 탭에 커서를 주고, 키보드 이동은 다음 탭으로 커서를 옮겨야 해서 부르는 쪽이 정합니다
탭과 패널이 모두 있어야 진행	둘 중 하나가 없는데 진행하면 탭 표시만 바뀌고 패널은 그대로인 어긋난 상태가 됩니다
모르는 이름은 `false`	오타로 모든 탭이 선택 해제되는 일을 막습니다

4. 시험 (실제 서버, 헤드리스 Chrome, 40개 통과)

`home.html`과 `join.html`에서 같은 시험을 했습니다.

확인	결과
시작 상태: Password 탭 선택, Email 패널 숨음	✓ ✓
`email` 선택: `aria-selected`, `tabindex`, 패널이 **실제로**(`getComputedStyle`) 바뀜	✓ ✓
이미 그 탭이면 같은 결과	✓ ✓
탭을 오가도 두 칸에 적은 값이 남음	✓ ✓
오류 문구도 건드리지 않음	✓ ✓
`focus` 인자가 없거나 `false`, `1`이면 커서를 옮기지 않음	✓ (6개)
`focus=true`면 고른 탭으로 커서	✓ ✓
숨겨질 패널의 칸에 있던 커서는 고른 탭으로 옮겨져 사라지지 않음	✓ ✓
같은 탭을 다시 고르면 칸의 커서를 그대로 둠	✓ ✓
모르는 이름 7종은 `false`이고 안 바뀜	✓ (14개)
모달이 `null`이면 `false`	✓ ✓
탭이나 패널이 없는 모달은 `false`이고 아무것도 안 바뀜	✓
내보내는 상수	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
`aria-selected`를 반대로	2개 실패
`tabindex` 갱신 제거	2개 실패
패널 전환 제거	2개 실패
입력칸 값 비우기	2개 실패
`focus`의 엄격한 비교를 느슨하게	2개 실패
`focus` 인자 무시	2개 실패
잃는 커서 처리 제거	**6개 실패**
탭만 있고 패널이 없는 경우를 진행	1개 실패
이름 검사 제거	**실패 없음**

마지막은 시험의 빈틈이 아니라 중복 방어입니다. 모르는 이름이면 `recover-tab-<이름>`인 요소가 없어서 바로 아래의 `chosen === null` 검사가 같은 경우를 `false`로 막기 때문입니다. 이름 검사는 요소를 찾으러 가기 전의 빠른 반환입니다.

5. 시험이 잡은 문제: 숨은 칸에서 커서가 사라짐

"이미 그 탭이면 같은 결과" 시험이 처음에 실패했습니다. 두 번 부른 상태를 비교했는데 한 가지만 달랐습니다. **커서(`document.activeElement`)였습니다.** 첫 호출 직후에는 `recover-password-email`이었다가, 잠시 뒤에는 아무 요소도 아닌 상태(`body`)가 되어 있었습니다.

- **원인:** Email 탭으로 바꾸면서 Password 패널을 숨겼는데, 커서가 그 패널의 이메일 칸에 있었습니다. 브라우저는 숨겨진 요소의 커서를 빼앗고(`body`로 보냄), 그것이 화면에 그려진 뒤에야 반영됩니다. 그래서 첫 시험 값은 옛 상태였습니다.
- **영향:** 키보드로 쓰는 사람이 Password 칸에 있다가 탭이 바뀌면 커서가 사라져 Tab 키를 눌러도 문서 맨 위부터 다시 시작합니다.
- **고침:** 패널을 숨기기 전에 "커서가 숨겨질 패널 안에 있는지" 확인하고, 그렇다면 고른 탭으로 커서를 옮깁니다. 시험도 같은 경우를 확인하게 더했고(위 표), 이 경우를 지우면 6개가 실패합니다.

6. 아직 이 함수를 부르는 곳이 없습니다

탭을 눌러도 아직 바뀌지 않습니다. 다음 함수가 클릭과 키보드(좌우 화살표, Home, End)를 이 함수에 연결합니다.

JS 167 — `initRecoverTabs()` (탭 클릭과 키보드 이동)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (내부 도우미 `tabNameOf()` 포함)
작업	P1-28 계정 찾기 화면: JS 세 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	166 `selectTab()`
다음	열기 연결: `Can't sign in?`, `home.html#recover`, `Back to Sign In`, 모달을 닫으면 로그인 화면으로 초기화

1. 하는 일

복구 화면의 탭 목록(`role="tablist"`)에 마우스와 키보드 조작을 연결해서, 이전 함수 `selectTab()`을 부르게 합니다.

조작	동작
탭을 누름	그 탭을 고름
→ (탭에 커서가 있을 때)	다음 탭. 끝에서는 처음으로 이어짐
←	이전 탭. 처음에서는 끝으로 이어짐
`Home`, `End`	첫 탭, 마지막 탭

옮기면 **바로 그 탭을 고르고 커서도 따라갑니다**(WAI-ARIA 탭 패턴의 자동 선택). 처리한 키만 기본 동작을 막고, `Ctrl`, `Alt`, `Meta`, `Shift`를 함께 누르면 처리하지 않습니다(브라우저와 화면 낭독기의 단축키를 가로채지 않기 위해서입니다).

2. 코드

```
function tabNameOf(tab) { /* id 가 recover-tab-<이름>이고 이름이 알려진 값이면 이름, 아니면 null */ }

export function initRecoverTabs(modal) {
  if (!modal) return false;
  const list = modal.querySelector('[role="tablist"]');
  if (!list) return false;

  list.onclick = function (event) {
    const tab = event.target && event.target.closest ? event.target.closest('[role="tab"]') : null;
    const name = tabNameOf(tab);
    if (name !== null) selectTab(modal, name);
  };

  list.onkeydown = function (event) {
    if (event.ctrlKey || event.altKey || event.metaKey || event.shiftKey) return;
    const key = event.key;
    if (key !== 'ArrowRight' && key !== 'ArrowLeft' && key !== 'Home' && key !== 'End') return;
    const current = event.target && event.target.closest ? event.target.closest('[role="tab"]') : null;
    if (tabNameOf(current) === null) return;
    /* names = 탭 이름 목록, index = 가려는 위치(→ 는 +1, ← 는 -1, 끝에서 이어짐, Home=0, End=마지막) */
```

```
    event.preventDefault();
    selectTab(modal, names[index], true);
  };
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initRecoverTabs = (modal) => {
  const list = modal?.querySelector('[role="tablist"]');
  if (!list) return false;

  list.onclick = (e) => {
    const name = tabNameOf(e.target.closest?.('[role="tab"]'));
    if (name) selectTab(modal, name);
  };
  list.onkeydown = (e) => {
    if (e.ctrlKey || e.altKey || e.metaKey || e.shiftKey) return;
    const names = [...list.querySelectorAll('[role="tab"]')].map(tabNameOf).filter(Boolean);
    const at = names.indexOf(tabNameOf(e.target.closest?.('[role="tab"]')));
    const next = {
      ArrowRight: (at + 1) % names.length,
      ArrowLeft: (at - 1 + names.length) % names.length,
      Home: 0,
      End: names.length - 1,
    }[e.key];
    if (at < 0 || next === undefined) return;
    e.preventDefault();
    selectTab(modal, names[next], true);
  };
  return true;
};
```

최신 문법	실제 코드	뜻
`modal?.querySelector(…)`	`if (!modal) return false;` 뒤에 조회	모달이 없으면 건너뜁니다
`e.target.closest?.(…)`	`event.target && event.target.closest ? … : null`	`closest`가 없는 대상도 안전하게 다룹니다
키별 값을 담은 객체 `{ ArrowRight: …, … }[e.key]`	`if / else if` 사슬	키에 따라 가려는 위치를 정합니다
`.map(…).filter(Boolean)`	`forEach`로 `names`에 담기	알려진 탭 이름만 모읍니다

문법 풀이

식	뜻
`(index + 1) % names.length`	나머지 연산으로 끝에서 처음으로 이어지게 합니다(2개면 0→1→0)
`(index - 1 + names.length) % names.length`	처음에서 이전으로 가면 음수가 되므로 길이를 더해 끝으로 이어지게 합니다
`event.target.closest('[role="tab"]')`	눌린 곳(탭 안의 글자 등)에서 가장 가까운 탭 버튼을 찾습니다
`list.onclick = …`	처리기를 대입합니다. 이미 있던 처리기를 바꿔서 두 번 연결해도 하나만 남습니다

3. 왜 이렇게

선택	이유
화살표로 옮기면 바로 선택 (자동 선택)	탭이 둘뿐이고 패널 전환이 가벼워서 "옮기고 Enter로 확정"하는 수동 방식보다 간단합니다
조합 키는 처리하지 않음	`Ctrl+→`는 글자 단위 이동, `Alt+←`는 브라우저 뒤로 가기 같은 기존 단축키입니다. 가로채면 안 됩니다
처리한 키만 `preventDefault()`	화살표가 화면을 스크롤하는 기본 동작은 막되, `a`나 `↑`는 그대로 둡니다
처리기를 **탭 목록 하나에만** 달고 이벤트 전달을 이용	탭마다 따로 달지 않아도 누른 탭을 `event.target`으로 알 수 있고, 탭을 나중에 늘려도 그대로 동작합니다. 입력칸은 탭 목록 밖에 있어서 칸 안의 화살표 키는 처리기에 닿지 않습니다
`onclick`, `onkeydown`에 대입	이 함수를 여러 번 불러도 처리기가 쌓이지 않게 하려는 습관입니다(홈 화면의 다른 함수들과 같음)
탭 이름은 `tabNameOf()`로 한 곳에서	`recover-tab-<이름>` 규칙을 한 곳에서만 해석합니다

4. 시험 (실제 서버, 마우스와 키보드 이벤트, 39개 통과)

헤드리스 Chrome에 **실제 마우스 클릭과 키 입력**(`Input.dispatchMouseEvent`, `Input.dispatchKeyEvent`)을 보냈고 `home.html`, `join.html`에서 같은 시험을 했습니다.

확인	결과
두 번 연결해도 `true`	✓ ✓
Email, Password 탭을 마우스로 누르면 선택되고 패널이 바뀜	✓ ✓ ✓ ✓
`→`: 다음 탭으로, 커서가 따라가고 `tabindex`도 바뀜	✓ ✓
`→`가 끝에서 처음으로, `←`가 처음에서 끝으로 이어짐, `←`로 돌아옴	✓ (6개)
`End`는 마지막 탭, `Home`은 첫 탭, 이미 첫 탭에서 `Home`을 눌러도 그대로	✓ (6개)
다른 키(`a`)는 처리하지 않음	✓ ✓
`Ctrl`, `Alt`, `Meta`, `Shift`와 함께 누른 `→`는 처리하지 않음	✓ (8개)
처리한 키(`→ ← Home End`)만 기본 동작을 막고 `a`, `↑`, `Ctrl+→`는 막지 않음	✓ ✓
입력칸에서 `→`를 눌러도 탭이 바뀌지 않음	✓ ✓
탭 목록의 빈 곳을 눌러도 바뀌지 않음	✓ ✓
모달이 `null`이거나 탭 목록이 없으면 `false`	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
클릭 처리 제거	2개 실패
끝에서 처음으로 안 이어지게	6개 실패
`Home`과 `End`를 뒤바꿈	16개 실패
옮긴 뒤 커서가 따라가지 않게	10개 실패
`preventDefault()` 제거	2개 실패
조합 키 검사 제거	6개 실패

바꾼 것	결과
`→`를 이전 탭 쪽으로 바꿈	실패 없음
탭 밖에서 온 키도 처리	실패 없음
처리기를 `addEventListener`로 두 번 연결	실패 없음

시험이 잡지 못한 세 가지, 모두 시험의 빈틈이 아니라 지금 구조의 성질입니다.

1. `→`**와** `←`**의 방향:** 탭이 **둘**이면 "다음"과 "이전"이 같은 탭이라 방향을 뒤바꿔도 결과가 같습니다. 탭이 셋 이상이 되면 방향이 드러나는데, 지금은 탭 이름 목록(`RECOVER_TABS`)에 둘뿐이라 만들어 시험할 수 없습니다. 탭을 늘릴 때 시험을 더해야 합니다.
2. **탭 밖의 키:** 처리기가 탭 목록에만 달려 있어서, 입력칸의 키 입력은 처리기에 도달하지 않습니다. 그 검사는 방어용 중복입니다.
3. **처리기 겹침:** 처리기를 두 번 달아도 둘 다 같은 `event.target`으로 **같은 탭을 고르므로** 결과가 같습니다(멱등). `onclick`, `onkeydown` 대입은 겹침을 줄이는 습관이지만 이 함수에서는 동작을 가르지 않습니다. 처음 시험에서 "처리기가 겹치면 한 바퀴 돌아 원위치"라고 가정했다가, 실제로는 같은 탭을 두 번 고르는 것뿐임을 확인하고 시험의 설명을 바로잡았습니다.

5. 아직 이 함수를 부르는 곳이 없습니다

복구 화면을 여는 연결(다음 함수)이 이 함수를 부릅니다. 그 전에는 복구 화면 자체가 열리지 않아서 탭을 쓸 수 없습니다.

JS 168 — `initRecover()` (계정 찾기 화면 열기와 닫기 연결)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (내부 도우미 `resetRecoverState()` 포함)
작업	P1-28 계정 찾기 화면: JS 네 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	167 `initRecoverTabs()`
다음	`home.html#recover` 주소로 복구 화면 열기, 그다음 요청 폼 전송

1. 하는 일

로그인 모달의 계정 찾기 화면을 실제 버튼과 모달의 닫힘에 연결합니다. 앞의 세 함수(화면 전환, 탭 고르기, 탭 조작)를 한곳에서 엮는 함수입니다.

조작	동작
`Can't sign in?`	복구 화면을 엽니다(처음 상태, Password 탭, 첫 입력칸에 커서)
`Back to Sign In`(복구 화면 아래, 보낸 뒤 화면)	로그인 화면으로 돌아오고 복구 화면 상태를 비웁니다
모달 닫기(`Close` 버튼, `Esc`, `close()`)	로그인 화면으로 되돌리고 복구 화면 상태를 비웁니다. **다음에 열면 항상 로그인 화면부터** 입니다
탭 클릭, 화살표	`initRecoverTabs()`를 여기서 연결

복구 화면 상태를 비운다는 것은 다음을 모두 처음으로 되돌린다는 뜻입니다: Password 탭 선택, 탭 목록과 `Back to Sign In` 줄이 보임, 보낸 뒤 화면이 숨음, 입력칸이 비워짐, 오류 문구가 숨음, 보낸 뒤 화면의 이메일 글자가 비워짐.

2. 코드 (요약)

```
function resetRecoverState(modal) {
  /* 탭 목록과 Back 줄 보이기, 보낸 뒤 화면 숨기기, Password 탭 선택,
     입력칸 비우기, 오류 문구 숨기기, 보낸 이메일 글자 비우기 */
}

export function initRecover(modal) {
  if (!modal || !modal.querySelector('[data-view="recover"]')) return false;

  const open = modal.querySelector('#login-recover');
  if (open) open.onclick = function () { resetRecoverState(modal); showView(modal, 'recover'); };

  for (const id of ['recover-back', 'recover-sent-back']) {
    const back = modal.querySelector('#' + id);
    if (back) back.onclick = function () { resetRecoverState(modal); showView(modal, 'login'); };
  }

  if (modal.recoverOnClose) modal.removeEventListener('close', modal.recoverOnClose);
  modal.recoverOnClose = function () { resetRecoverState(modal); showView(modal, 'login'); };
```

```
  modal.addEventListener('close', modal.recoverOnClose);

  initRecoverTabs(modal);
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initRecover = (modal) => {
  if (!modal?.querySelector('[data-view="recover"]')) return false;
  const go = (view) => { resetRecoverState(modal); showView(modal, view); };

  modal.querySelector('#login-recover')?.addEventListener('click', () => go('recover'));
  ['recover-back', 'recover-sent-back'].forEach((id) =>
    modal.querySelector(`#${id}`)?.addEventListener('click', () => go('login')));
  modal.addEventListener('close', () => go('login'));

  initRecoverTabs(modal);
  return true;
};
```

최신 문법	실제 코드	뜻
`modal?.querySelector(…)`	`!modal || !modal.querySelector(…)`	모달이 없으면 건너뜁니다
`…?.addEventListener(…)`	`if (open) open.onclick = …`	버튼이 있을 때만 연결합니다
`go(view)` 도우미	같은 두 줄을 반복	비우고 화면을 바꾸는 일을 한 번만 적습니다
`addEventListener` 한 번	`onclick` 대입, `close`는 이전 처리기를 떼고 다시 닮	여러 번 불러도 처리기가 쌓이지 않게 합니다

문법 풀이

식	뜻
`modal.recoverOnClose`	모달 요소에 처리기를 붙여 두어, 다시 부를 때 이전 것을 `removeEventListener`로 뗄 수 있게 합니다
`for (const id of […])`	같은 동작을 두 버튼에 연결합니다
`close` 이벤트	`<dialog>`가 닫힐 때(`Close` 버튼, `Esc`, `close()`) 한 번 일어납니다

3. 왜 이렇게

선택	이유
닫을 때 입력값을 비움	복구 화면에 적은 이메일은 개인정보입니다. 모달을 닫아 두면 같은 브라우저를 쓰는 다음 사람이 그 값을 볼 수 있어서, 닫는 순간 비웁니다. 시험에서 확인했습니다
열 때와 돌아올 때 모두 초기화	두 곳이 겹치지만 한쪽만 있으면, 다른 경로(나중에 만들 `#recover` 주소로 열기 등)가 생겼을 때 이전 상태가 남을 수 있습니다. 한쪽을 지워도 시험은 통과하고, **둘 다 지우면 실패**합니다(아래 §4)
닫으면 로그인 화면으로 되돌림	다음에 `Sign In`을 눌렀을 때 복구 화면이 먼저 보이면 사용자가 당황합니다

선택	이유
화면 전환과 비우기를 항상 함께 부름	따로 부르다가 하나를 빼먹는 일을 막습니다
버튼이 없어도 오류 없이 진행	이 모달 조각은 여러 페이지가 쓰고, 일부 페이지가 버튼을 빼도 다른 부분이 동작해야 합니다
`close` 처리기는 모달에 붙여 둔 것을 떼고 다시 닮	`login-modal`은 다른 곳(`initLoginModalFromHash`)도 `close`를 듣고 있어서 `onclose`로 덮어쓰면 그쪽이 사라집니다. `addEventListener`를 쓰되, 이 함수를 두 번 불러도 겹치지 않게 했습니다

4. 시험 (실제 서버, 마우스와 키보드 이벤트, 28개 통과)

헤드리스 Chrome에 **실제 마우스 클릭과 `Esc` 키**를 보냈고 `home.html`, `join.html`에서 같은 시험을 했습니다.

확인	결과
연결하면 `true`(두 번 불러도)	✓ ✓
모달을 열면 로그인 화면(제목 `Sign In`)	✓ ✓
`Can't sign in?`을 누르면 복구 화면, 제목, Password 탭, 첫 칸에 커서	✓ ✓
탭 연결도 됨(Email 탭을 누르면 선택)	✓ ✓
`Back to Sign In`: 로그인 화면, 제목, 로그인 칸에 커서	✓ ✓
다시 열면 처음 상태(Password 탭, 입력값과 오류 문구가 비워짐)	✓ ✓
보낸 뒤 화면의 `Back to Sign In`: 로그인 화면으로	✓ ✓
다시 열면 보낸 뒤 화면이 사라지고 탭과 Back 줄이 돌아옴, 이메일 글자 비워짐	✓ ✓
복구 화면에서 `close()`로 닫았다 다시 열면 로그인 화면, 입력값 비워짐	✓ ✓
`Esc` 키로 닫았다 다시 열어도 로그인 화면, 입력값 비워짐	✓ ✓ ✓ ✓
로그인 화면(폼, 입력칸, `Join Us`)은 그대로	✓ ✓
모달이 `null`이거나 복구 화면이 없으면 `false`	✓
복구 화면은 있지만 버튼, 탭이 없어도 오류 없이 `true`	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
열기 연결 제거	12개 실패
`Back to Sign In` 연결 제거	8개 실패
닫기 처리 제거	6개 실패
닫을 때 입력값을 안 비움	6개 실패
닫을 때 로그인 화면으로 안 돌림	6개 실패
탭 연결 제거	2개 실패
보낸 뒤 화면 초기화 제거	2개 실패
탭 목록 복구 제거	2개 실패
오류 문구를 안 비움	2개 실패
복구 화면이 없어도 진행	1개 실패
열 때의 초기화만 제거	**실패 없음**
돌아올 때의 초기화만 제거	**실패 없음**

바꾼 것	결과
위 둘을 **함께** 제거	4개 실패

마지막 세 줄은 시험의 빈틈이 아니라 **이중 방어**의 증거입니다. 한쪽(열 때 또는 돌아올 때)만 지우면 다른 쪽이 같은 일을 하므로 결과가 같고, 둘 다 지우면 실패합니다. 그리고 **닫을 때**의 초기화가 세 번째 방어입니다. 일부러 겹쳐 두었습니다.

5. 아직 이 함수를 부르는 곳이 없습니다

이 함수를 `home.js`와 `join.js`가 불러야 실제 화면에서 동작합니다. 연결은 `#recover` 주소로 여는 함수까지 만든 뒤에 한 번에 하겠습니다(두 번 손대지 않기 위해서). 지금까지의 함수 4개는 모두 시험으로만 확인했습니다.

JS 169 — `initRecoverFromHash()` (주소 `#recover`로 복구 화면 열기)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js`
작업	P1-28 계정 찾기 화면: JS 다섯 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	168 `initRecover()`
다음	`home.js`, `join.js`에 연결, 그다음 요청 폼 전송

1. 하는 일

주소가 `#recover`이면 로그인 모달을 열고 **복구 화면**을 보여 줍니다. 이메일 안내 메일의 `Reset password` 버튼(`home.html#recover`)이 이 주소를 가리킵니다. 로그인 모달의 `#login` 주소와 같은 방식입니다.

상황	동작
`home.html#recover`로 들어옴	모달이 열리고 복구 화면(Password 탭, 제목 `Can't sign in?`)
페이지 안에서 주소가 `#recover`로 바뀜(`hashchange`)	같은 동작
이미 열린 로그인 화면에서 `#recover`로 바뀜	복구 화면으로 바뀜
모달을 닫음	주소의 `#recover`를 지움(새로 고침하면 다시 열리는 일을 막음)
`#recovery`, `#recoverx`, `#contact`, 해시 없음	복구 화면을 열지 않음

2. 코드

```
export function initRecoverFromHash(modal) {
  if (!modal || !modal.querySelector('[data-view="recover"]')) return false;

  if (modal.recoverHashHandler) window.removeEventListener('hashchange', modal.recoverHashHandler);
  if (modal.recoverHashOnClose) modal.removeEventListener('close', modal.recoverHashOnClose);

  const openIfRecoverHash = function () {
    if (location.hash !== '#recover') return;
    if (!modal.open) modal.showModal();
    resetRecoverState(modal);
    showView(modal, 'recover');
  };
  modal.recoverHashHandler = openIfRecoverHash;
  modal.recoverHashOnClose = function () {
    if (location.hash === '#recover') {
      history.replaceState(null, '', location.pathname + location.search);
    }
  };

  window.addEventListener('hashchange', openIfRecoverHash);
  modal.addEventListener('close', modal.recoverHashOnClose);
  openIfRecoverHash();
```

```
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initRecoverFromHash = (modal) => {
  if (!modal?.querySelector('[data-view="recover"]')) return false;
  const openIfRecoverHash = () => {
    if (location.hash !== '#recover') return;
    if (!modal.open) modal.showModal();
    resetRecoverState(modal);
    showView(modal, 'recover');
  };
  window.addEventListener('hashchange', openIfRecoverHash);
  modal.addEventListener('close', () => {
    if (location.hash === '#recover') history.replaceState(null, '', location.pathname + location.searc
h);
  });
  openIfRecoverHash();
  return true;
};
```

최신 문법	실제 코드	뜻
`modal?.querySelector(…)`	`!modal || !modal.querySelector(…)`	모달이 없으면 건너뜁니다
처리기를 바로 `addEventListener`	모달 요소에 붙여 두었다가 이전 것을 떼고 다시 닮	이 함수를 두 번 불러도 처리기가 쌓이지 않습니다

문법 풀이

식	뜻
`location.hash !== '#recover'`	**정확히** `#recover`일 때만 엽니다. `#recovery`나 `#recoverx`는 열지 않습니다(`startsWith`가 아니라 같음 비교)
`hashchange`	주소의 `#…` 부분이 바뀔 때(링크 클릭, 직접 입력) 일어납니다
`history.replaceState(null, '', …)`	새 기록을 쌓지 않고 주소만 바꿉니다. 해시를 지울 때 뒤로 가기 단추가 `#recover`로 돌아가지 않게 합니다
`location.pathname + location.search`	경로와 `?…` 부분은 그대로 두고 `#…`만 뺍니다

3. 왜 이렇게

선택	이유
정확히 `#recover`일 때만	비슷한 해시(`#recovery`)가 다른 곳(나중에 생길 앵커)에 쓰일 수 있고, `#login?next=…` 같은 값을 붙이는 `#login`과 달리 `#recover`는 값을 받지 않아서 더 엄격하게 비교해도 됩니다
열 때 `resetRecoverState()`를 거침	`hashchange`로 이미 입력 중인 복구 화면을 다시 열 수 있으므로 항상 깨끗한 처음 상태에서 시작합니다
모달을 닫으면 `#recover`를 지움	지우지 않으면 닫은 뒤 새로 고침할 때 복구 화면이 다시 열립니다. `#login`을 지우는 홈 화면의 기존 함수와 같은 이유입니다

선택	이유
다른 해시로 바뀐 뒤 닫으면 **그 해시를 지우지 않음**	사용자가 `#feature`로 이동했는데 모달을 닫으면서 위치 정보가 사라지면 안 됩니다. 시험으로 확인했습니다
닫기 처리기를 `initRecover()`의 것과 **따로 둠**	두 함수가 서로 몰라도 각자 동작하고, 한쪽만 써도 됩니다
모달이 이미 열려 있으면 `showModal()`을 부르지 않음	옛 브라우저는 이미 열린 `<dialog>`에 `showModal()`을 부르면 오류를 냅니다

4. 시험 (실제 서버, 31개 통과)

`home.html`, `join.html` 두 페이지에서 같은 시험을 했습니다.

확인	결과
연결하면 `true`(세 번 불러도)	✓ ✓
`#recover`로 들어오면 모달이 열리고 복구 화면, 제목, Password 탭	✓ ✓
닫으면 주소의 `#recover`가 지워지고 경로는 그대로	✓ ✓
다시 열면 로그인 화면(`#recover`가 남아 있지 않음)	✓ ✓
페이지 안에서 주소가 `#recover`로 바뀌면 열림	✓ ✓
`Esc`로 닫아도 `#recover`가 지워짐	✓ ✓
이미 열린 로그인 화면에서 `#recover`로 바뀌면 복구 화면으로, 오류 없음	✓ ✓
`#recovery`, `#recoverx`, `#contact`, `#`은 복구 화면을 열지 않음 (`#login`은 홈 화면이 원래 로그인 모달을 여는 주소라 모달은 열릴 수 있지만 복구 화면은 아님)	✓ (10개)
다른 해시(`#feature`)로 바뀐 뒤 닫으면 그 해시를 지우지 않음	✓ ✓
해시가 없으면 연결해도 열지 않음	✓ ✓
연결 뒤 흐름 전체에서 자바스크립트 오류 없음	✓ ✓
모달이 `null`이거나 복구 화면이 없으면 `false`	✓

시험이 잡은 시험의 착오. 처음 시험은 `#login`, `#contact`가 "모달을 열지 않음"이라고 가정했는데, `home.html`에는 `#login` 해시로 로그인 모달을 여는 기존 함수가 있어서 `#login`은 모달이 열렸습니다. 그리고 앞 시험이 열어 둔 모달이 남아서 다음 해시의 결과에 섞였습니다. 이 함수의 오류가 아니라 시험의 가정이 틀린 것이어서, 시험을 "복구 화면을 열지 않음"으로 바로잡고 각 해시 사이에 모달을 닫도록 했습니다.

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
처음 확인 제거(`#recover`로 들어와도 안 열림)	4개 실패
`hashchange` 연결 제거	8개 실패
닫을 때 해시를 지우지 않음	10개 실패
`#recover`를 `startsWith`로 비교	4개 실패
열린 뒤 복구 화면으로 바꾸지 않음	6개 실패
닫을 때 항상 해시를 지움(`#feature`도)	2개 실패
모든 해시에서 열림	12개 실패
복구 화면이 없어도 진행	1개 실패

바꾼 것	결과
`!modal.open` 검사 제거(`showModal()`을 항상 부름)	**실패 없음**

마지막은 시험의 빈틈이 아니라 브라우저의 성질입니다. 지금 쓰는 Chrome은 이미 열린 모달에 `showModal()`을 불러도 아무 일도 하지 않고 오류를 내지 않습니다. 이 검사는 오류를 내는 옛 브라우저를 위한 방어입니다.

5. 아직 이 함수를 부르는 곳이 없습니다

`home.js`와 `join.js`가 `initRecover()`와 이 함수를 불러야 실제 화면에서 동작합니다. 다음 단계에서 두 줄을 더합니다.

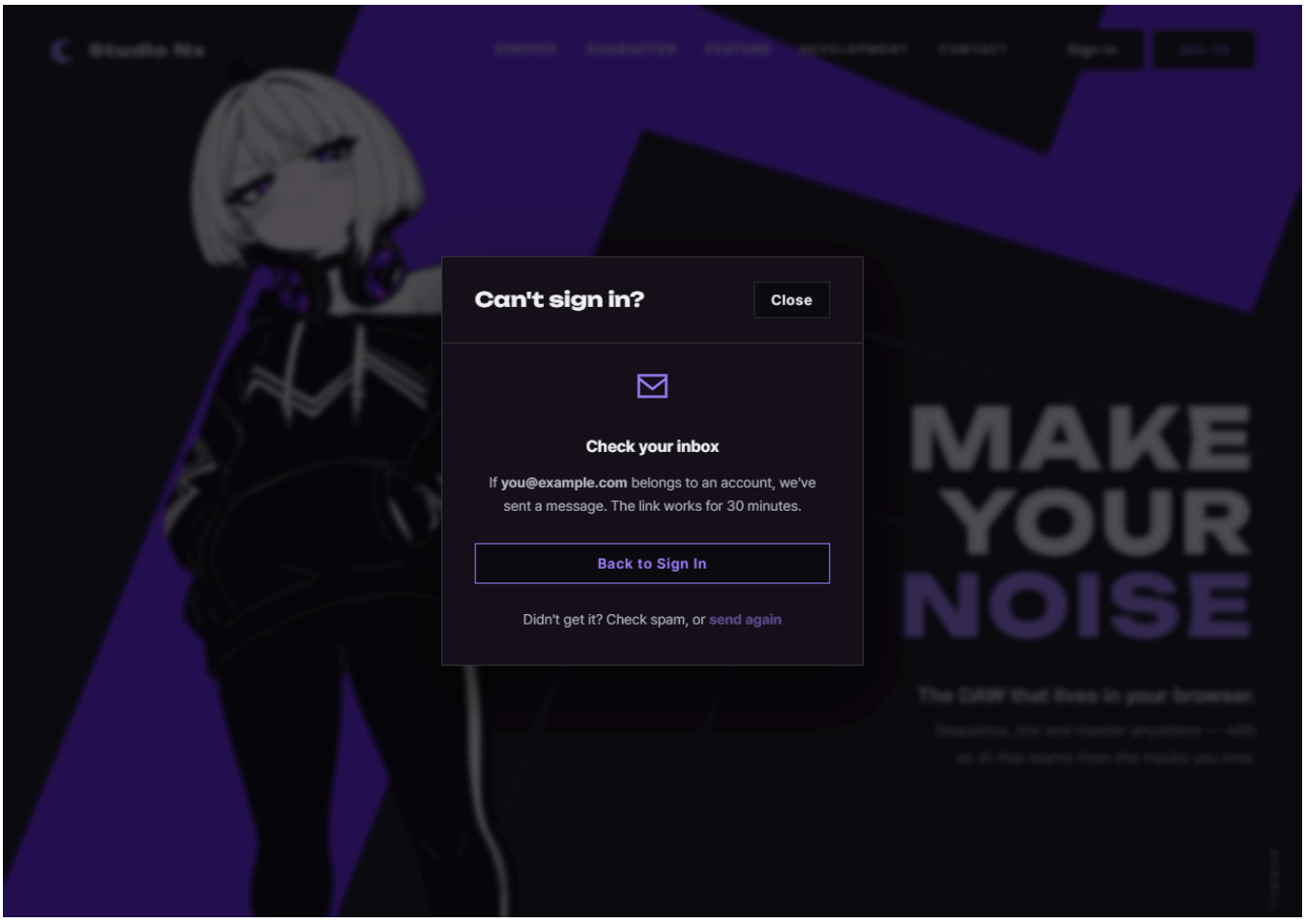
JS 170 — `showSentView()` ("메일을 보냈다" 화면 보이기)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js`
작업	P1-28 계정 찾기 화면: JS 여섯 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	169 `initRecoverFromHash()`
다음	`initRecoverForms()`: 두 폼의 전송을 맡아 API를 부르고 이 함수를 부름

1. 하는 일

복구 화면에서 요청이 성공한 뒤 보여 주는 **"Check your inbox" 화면**으로 바꿉니다.

하는 일	설명
화면 정리	탭 목록, 두 패널, `Back to Sign In` 줄을 숨기고 보낸 뒤 화면(`#recover-sent`)을 보입니다
이메일 표시	입력한 이메일을 `#recover-sent-email`에 **글자로** 넣습니다
보충 문장	Password 탭은 "The link works for 30 minutes.", Email 탭은 "If you don't see it, check your spam folder."
커서	`Back to Sign In` 버튼으로 옮깁니다(입력칸이 숨었으므로)
반환	바꿨으면 `true`, 모르는 `kind`, 모달이나 복구 화면, 보낸 뒤 화면이 없으면 아무것도 바꾸지 않고 `false`



2. 코드

```
const SENT_DETAIL = {
  password: 'The link works for 30 minutes.',
  email: "If you don't see it, check your spam folder.",
};

export function showSentView(modal, kind, email) {
  if (!modal || RECOVER_TABS.indexOf(kind) < 0) return false;
  const recover = modal.querySelector('[data-view="recover"]');
  const sent = modal.querySelector('#recover-sent');
  if (!recover || !sent) return false;

  const tabs = recover.querySelector('[role="tablist"]');
  const backRow = recover.querySelector('#recover-back-row');
  if (tabs) tabs.hidden = true;
  for (const panel of recover.querySelectorAll('[role="tabpanel"]')) panel.hidden = true;
  if (backRow) backRow.hidden = true;
  sent.hidden = false;

  const shown = sent.querySelector('#recover-sent-email');
  if (shown) shown.textContent = typeof email === 'string' ? email : '';
  const detail = sent.querySelector('#recover-sent-detail');
  if (detail) detail.textContent = SENT_DETAIL[kind];

  showView(modal, 'recover');
  const back = sent.querySelector('#recover-sent-back');
  if (back) back.focus();
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const showSentView = (modal, kind, email) => {
  if (!modal || !RECOVER_TABS.includes(kind)) return false;
  const recover = modal.querySelector('[data-view="recover"]');
  const sent = modal.querySelector('#recover-sent');
  if (!recover || !sent) return false;

  recover.querySelector('[role="tablist"]')?.setAttribute('hidden', '');
  recover.querySelectorAll('[role="tabpanel"]').forEach((p) => { p.hidden = true; });
  recover.querySelector('#recover-back-row')?.setAttribute('hidden', '');
  sent.hidden = false;

  sent.querySelector('#recover-sent-email')?.replaceChildren(typeof email === 'string' ? email : '');
  sent.querySelector('#recover-sent-detail')?.replaceChildren(SENT_DETAIL[kind]);

  showView(modal, 'recover');
  sent.querySelector('#recover-sent-back')?.focus();
  return true;
};
```

최신 문법	실제 코드	뜻
`…?.setAttribute('hidden', '')`	`if (tabs) tabs.hidden = true;`	요소가 있을 때만 숨깁니다
`replaceChildren(글자)`	`textContent = 글자`	안의 내용을 글자로 바꿉니다. 둘 다 마크업으로 해석하지 않습니다
`?.focus()`	`if (back) back.focus();`	버튼이 있을 때만 커서를 줍니다

문법 풀이

식	뜻
`shown.textContent = …`	값을 **글자 그대로** 넣습니다. `innerHTML`은 `<img onerror=…>` 같은 값을 마크업으로 해석해서 스크립트가 실행될 수 있습니다
`typeof email === 'string' ? email : ''`	문자열이 아니면(숫자, `undefined`, 객체) 빈 글자로 둡니다. 화면에 `undefined`가 찍히는 것을 막습니다
`SENT_DETAIL[kind]`	요청 종류에 맞는 문장을 고릅니다

3. 왜 이렇게

선택	이유
`textContent`로 이메일을 넣음	이메일은 사용자가 입력한 값이라 믿을 수 없습니다. `a"><img src=x onerror=…>@e.com` 같은 값이 마크업이 되어 스크립트가 실행되는 일(XSS)을 막습니다. 시험에서 `onerror`가 실행되지 않음을 확인했습니다
문구는 "있다면 보냈다"	서버 응답이 가입 여부와 관계없이 항상 같으므로 화면도 같은 말만 합니다. 이메일을 보여 주는 것은 사용자가 오타를 알아채게 하려는 것이고, 가입 여부를 알려 주지 않습니다
보충 문장을 종류별로	Password 탭의 메일에는 링크 유효 시간(30분)이 의미 있고, Email 탭의 메일에는 의미가 없어서 스팸함 안내로 대신합니다
커서를 `Back to Sign In`으로	입력칸이 모두 숨으므로 커서가 사라지면(§166의 같은 문제) 키보드 사용자가 위치를 잃습니다
로그인 화면에서 불러도 복구 화면으로 바꿈	이 함수가 "복구 화면 안에서만 불러야 한다"는 전제를 부르는 쪽에 요구하지 않게 합니다
모르는 `kind`는 `false`	오타로 보충 문장이 `undefined`가 되는 일을 막습니다
이메일을 비우지 않고 계속 보여 줌(돌아가면 비움)	돌아가는 처리(`initRecover`)가 이미 비웁니다. 이 함수는 보여 주는 일만 합니다

4. 시험 (실제 서버, 36개 통과)

`home.html`, `join.html`에서 같은 시험을 했습니다.

확인	결과
`password`: 보낸 뒤 화면만 보임(탭, 두 패널, Back 줄이 **실제로** 숨음), 로그인 화면에서 불러도 바뀜	✓ ✓
이메일, 보충 문장(30분), 제목, 커서가 `Back to Sign In`	✓ ✓
`email`: 보충 문장이 스팸함 안내로, 이메일 글자 갱신	✓ ✓
같은 값으로 다시 불러도 같은 결과	✓ ✓

확인	결과
이메일의 `<`, `&`는 마크업이 되지 않고 `onerror`가 실행되지 않음	✓ ✓
이메일이 문자열이 아니면(`undefined`, `123`) 빈 글자	✓ ✓ ✓ ✓
`Back to Sign In`을 누르면 로그인 화면, 보낸 뒤 화면과 이메일이 비워짐	✓ ✓
다시 열면 탭과 Back 줄이 돌아오고 보낸 뒤 화면은 숨음	✓ ✓
모르는 `kind` 7종은 `false`이고 안 바뀜	✓ (14개)
모달이 `null`이면 `false`	✓ ✓
보낸 뒤 화면이나 복구 화면이 없으면 `false`	✓
세부 요소(이메일 칸, 버튼)가 없어도 오류 없이 `true`	✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
탭 목록을 안 숨김	2개 실패
패널을 안 숨김	2개 실패
Back 줄을 안 숨김	2개 실패
보낸 뒤 화면을 안 보이게	7개 실패
이메일을 `innerHTML`로 넣음(XSS)	2개 실패
문자열이 아니어도 그대로 넣음	2개 실패
보충 문장을 고정	2개 실패
커서 이동 제거	2개 실패
화면 전환(`showView`) 제거	4개 실패
`kind` 검사 제거	14개 실패
보낸 뒤 화면이 없어도 진행	시험이 오류로 중단됨(없는 요소를 읽어서 예외)

모든 변이가 시험에 걸렸습니다.

5. 아직 이 함수를 부르는 곳이 없습니다

다음 함수(`initRecoverForms`)가 폼 전송이 성공하면 이 함수를 부릅니다.

JS 171 — `initRecoverForms()` (복구 화면의 두 폼 전송)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (내부 도우미 `recoverErrorText()`, 상수 `RECOVER_FORMS` 포함)
작업	P1-28 계정 찾기 화면: JS 일곱 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	170 `showSentView()`
다음	`send again` 1분 잠금

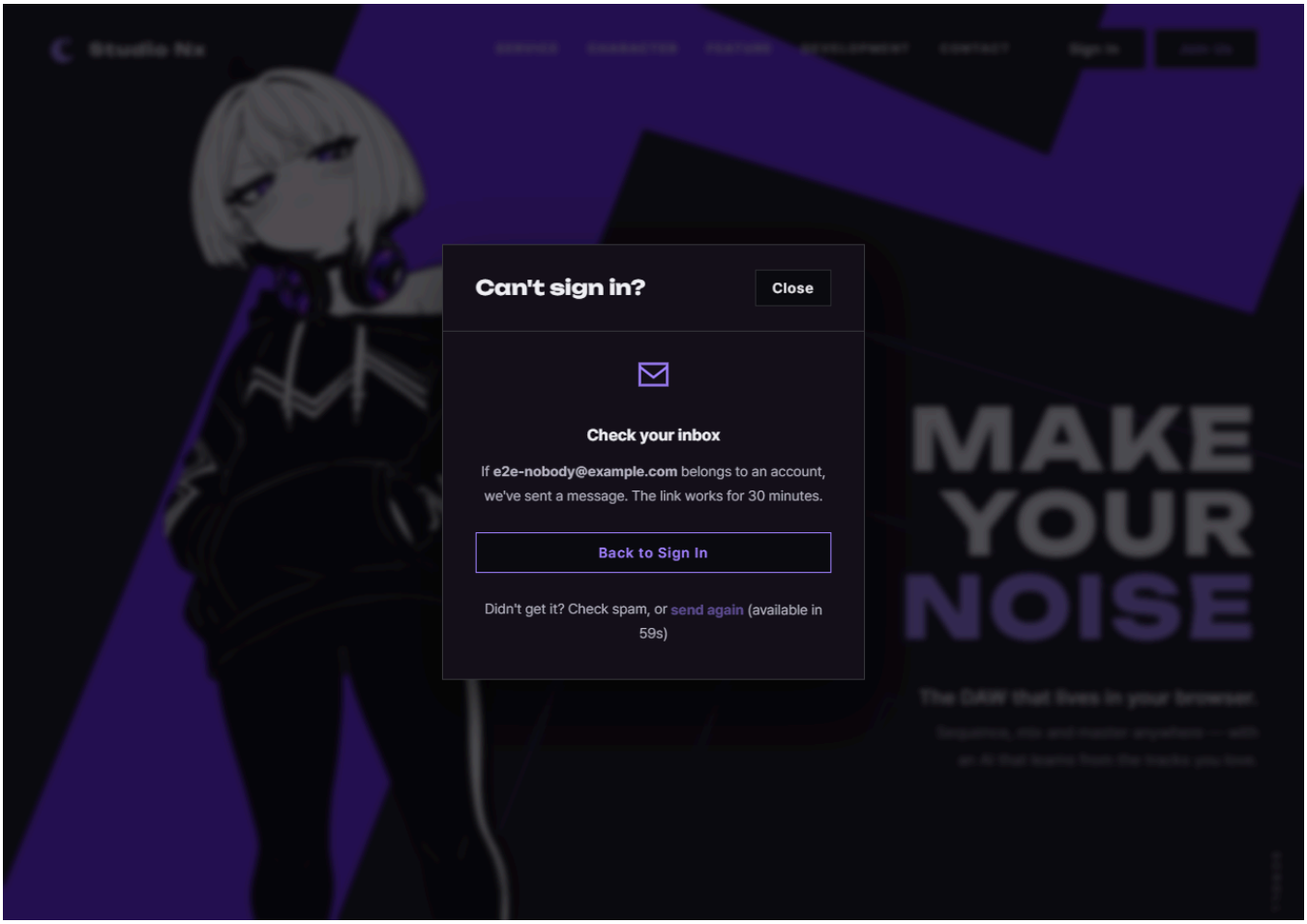
1. 하는 일

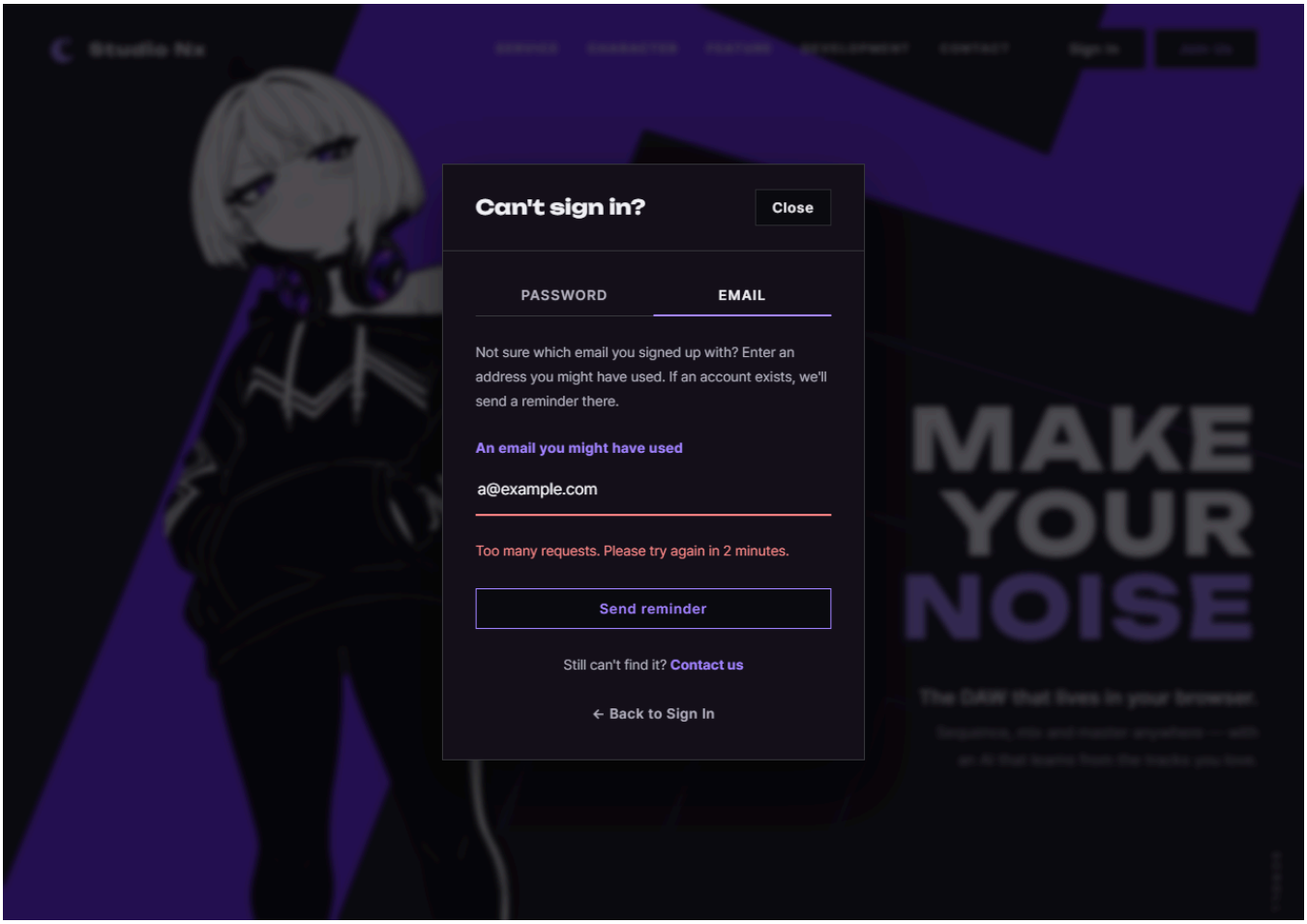
복구 화면의 두 폼(Password 탭, Email 탭)이 전송되면 서버 API를 부르고 결과를 화면에 보여 줍니다.

탭	보내는 곳
Password	`POST /api/v1/auth/password-reset/request`
Email	`POST /api/v1/auth/email-reminder/request`

결과	화면
성공(202)	`showSentView()`로 "Check your inbox" 화면. 성공한 요청의 종류와 이메일을 `modal.recoverLast`에 남김 (`send again`이 쓸 값)
429(요청이 잦음)	폼 아래에 "Too many requests. Please try again in N minute(s)." (`retry_after`를 분으로 올림, 없으면 10분)
422(형식 오류)	"Please enter a valid email address."
연결 실패	"Can't reach the server. Check your connection and try again."
그 밖(500 등)	"Something went wrong. Please try again."

실패하면 입력칸에 `aria-invalid="true"`를 달고 커서를 돌려줍니다. 새로 전송하거나 입력칸에 다시 적기 시작하면 오류 표시가 사라집니다.


Can't sign in?
Close
Check your inbox
If e2e-nobody@example.com belongs to an account, we've sent a message. The link works for 30 minutes.
Back to Sign In
Didn't get it? Check spam, or send again (available in 59s)
MAKE
YOUR
NOISE


Can't sign in?
Close
PASSWORD
EMAIL
Not sure which email you signed up with? Enter an address you might have used. If an account exists, we'll send a reminder there.
An email you might have used
a@example.com
Too many requests. Please try again in 2 minutes.
Send reminder
Still can't find it? Contact us
← Back to Sign In
MAKE
YOUR
NOISE

2. 코드 (요약)

```
const RECOVER_FORMS = [
  { kind: 'password', formId: 'recover-password-form', errorId: 'recover-password-error', path: '/auth/password-reset/request' },
  { kind: 'email',    formId: 'recover-email-form',    errorId: 'recover-email-error',    path: '/auth/email-reminder/request' },
];

function recoverErrorText(err) { /* 429, 422, 연결 실패(0), 그 밖을 고정 문구로 */ }

export function initRecoverForms(modal) {
  if (!modal) return false;
  let wired = false;
  for (const spec of RECOVER_FORMS) {
    const form = modal.querySelector('#' + spec.formId);
    if (!form) continue;
    const input = form.querySelector('input[name="email"]');
    const button = form.querySelector('button[type="submit"]');
    const errorBox = modal.querySelector('#' + spec.errorId);
    if (!input) continue;
    wired = true;

    const clearError = function () { /* 문구 비우고 숨기기, aria-invalid 풀기 */ };
    input.oninput = clearError;
    form.onsubmit = async function (event) {
      event.preventDefault();
      if (form.getAttribute('data-busy') === 'true') return;   // 연속 전송 무시
      form.setAttribute('data-busy', 'true');
      if (button) button.disabled = true;
      clearError();
      const email = input.value.trim();
      try {
        await api.post(spec.path, { email: email });
        modal.recoverLast = { kind: spec.kind, email: email };
        showSentView(modal, spec.kind, email);
      } catch (err) {
        /* 문구 보이기, aria-invalid, 입력칸에 커서 */
      } finally {
        form.removeAttribute('data-busy');
        if (button) button.disabled = false;
      }
    };
  }
  return wired;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initRecoverForms = (modal) => {
  if (!modal) return false;
  let wired = false;
  for (const { kind, formId, errorId, path } of RECOVER_FORMS) {
    const form = modal.querySelector(`#${formId}`);
    const input = form?.querySelector('input[name="email"]');
```

```
    if (!input) continue;
    wired = true;
    const button = form.querySelector('button[type="submit"]');
    const errorBox = modal.querySelector(`#${errorId}`);
    const clearError = () => { errorBox.textContent = ''; errorBox.hidden = true; input.removeAttribute
('aria-invalid'); };

    input.addEventListener('input', clearError);
    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      if (form.dataset.busy) return;
      form.dataset.busy = 'true';
      button.disabled = true;
      clearError();
      const email = input.value.trim();
      try {
        await api.post(path, { email });
        modal.recoverLast = { kind, email };
        showSentView(modal, kind, email);
      } catch (err) {
        errorBox.textContent = recoverErrorText(err);
        errorBox.hidden = false;
        input.setAttribute('aria-invalid', 'true');
        input.focus();
      } finally {
        delete form.dataset.busy;
        button.disabled = false;
      }
    });
  }
  return wired;
};
```

최신 문법	실제 코드	뜻
`for (const { kind, … } of …)`	`for (const spec of …)` 후 `spec.kind`	객체의 값을 바로 풀어 씁니다(구조 분해)
`{ email }`	`{ email: email }`	이름이 같으면 줄여 씁니다
`form.querySelector(…)?.…`	`if (!form) continue;`	폼이 없으면 건너뜁니다
`form.dataset.busy`	`getAttribute('data-busy')`	`data-` 속성을 읽습니다
`addEventListener('submit', …)`	`form.onsubmit = …`	처리기를 연결합니다. 이 코드는 두 번 불러도 겹치지 않도록 대입을 씁니다

문법 풀이

식	뜻
`async function` / `await api.post(…)`	서버 응답을 기다립니다. 실패하면 `catch`로 갑니다
`finally`	성공이든 실패든 마지막에 한 번 실행되어 버튼을 풉니다
`err.status`	`api.js`의 `ApiError`에 있는 HTTP 상태(네트워크 오류는 0)
`Math.ceil(wait / 60)`	초를 분으로 **올림**합니다(61초는 2분: 1분 뒤에 눌러도 아직 막혀 있을 수 있으므로)

3. 왜 이렇게

선택	이유
서버가 보낸 문구를 그대로 보여 주지 않음	서버 문구에 내부 사정이 섞일 수 있고, 가입 여부를 짐작하게 하는 문구가 새면 안 됩니다. 화면은 상태 코드로 고정 문구를 고릅니다. 시험에서 서버 문구에 `SECRET`을 넣어 노출되지 않음을 확인했습니다
성공하면 **항상 같은 화면**	서버 응답이 가입 여부와 관계없이 같으므로 화면도 같습니다
429를 **올림**으로	61초를 1분으로 버리면 사용자가 1분 뒤에 눌러도 또 막혀서 "안내가 틀렸다"고 느낍니다
보내는 동안 버튼 잠금, 연속 전송 무시	연속 클릭으로 같은 메일이 두 통 가거나 횟수 제한(10분에 3번)을 헛되이 쓰는 일을 막습니다. 버튼만 잠그면 Enter 키로 보내는 경우가 샐 수 있어서 `data-busy`로 폼 자체도 막습니다
실패하면 **입력칸으로 커서**	오류를 고치고 다시 보내려면 입력칸에 있어야 합니다. `aria-invalid`는 화면 낭독기에 알립니다
다시 적으면 오류를 지움	이미 고치기 시작한 사람에게 옛 오류가 남아 있으면 혼란스럽습니다
이메일 형식 검사는 브라우저(`type="email"`)에 맡김	틀리면 `initFormAlerts`가 알림창을 띄우고 이 처리기는 불리지 않습니다. 서버의 422는 그래도 올 수 있어서 문구를 둡니다
성공한 요청을 `modal.recoverLast`에 저장	`send again`이 같은 종류와 이메일로 다시 보낼 수 있게 합니다(다음 함수)
`api.post`를 씀	다른 화면과 같은 호출 모듈입니다. 오류를 `ApiError`로 통일하고 네트워크 실패는 상태 0으로 줍니다

4. 시험 (`fetch`를 가짜로 바꿔서 53개, 실제 서버로 2개 통과)

실제 서버에는 요청 횟수 제한(이메일 3번, IP 10번)과 메일 기록이 있어서, 반복 시험은 브라우저의 `fetch`를 가짜로 바꾸고 보낸 요청과 화면을 확인했습니다. `home.html`, `join.html`에서 같은 시험을 했습니다.

확인	결과
연결하면 `true`(두 번 불러도)	✓ ✓
Password 탭: 올바른 주소, `POST`, 공백 없는 이메일, JSON	✓ ✓
한 번만 보냄, 보낸 뒤 화면(30분 문장, 이메일), 커서가 Back 버튼	✓ (6개)
성공한 요청의 종류와 이메일이 `modal.recoverLast`에 남음	✓ ✓
Email 탭: 이메일 안내 주소, 스팸함 문장	✓ ✓
실패 7종(429가 61초, 600초, 1초, 값 없음, 422, 500, 연결 실패)이 정해진 문구이고, 보낸 뒤 화면은 안 뜨고, **서버 문구가 노출되지 않음**	✓ (14개)
실패하면 커서가 입력칸으로, `aria-invalid`, 버튼이 다시 켜짐	✓ (14개)
Email 탭의 실패는 Email 탭의 문구 자리에	✓ ✓
다시 입력하면 오류 문구와 `aria-invalid`가 사라짐	✓ ✓
보내는 동안 버튼이 잠기고 연속 세 번 전송해도 요청은 한 번, 끝나면 풀림	✓ (4개)
오류 뒤 다시 보내 성공하면 이전 오류가 지워짐	✓ ✓
형식이 틀린 이메일은 서버로 가지 않음	✓ ✓
모달이 `null`이거나 폼이나 이메일 칸이 없으면 `false`	✓
실제 서버: 없는 주소로 요청해도 같은 "보냈다" 화면, 자바스크립트 오류 없음	✓ ✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

바꾼 것	결과
Email 탭도 Password 탭과 같은 주소로	2개 실패
성공해도 보낸 뒤 화면을 안 띄움	8개 실패
`recoverLast`를 남기지 않음	시험이 예외로 중단
서버 문구를 그대로 노출	16개 실패
429를 올림 대신 버림	4개 실패
429의 기본값(10분) 제거	2개 실패
버튼 잠금 제거	2개 실패
연속 전송 무시(`data-busy`) 제거	4개 실패
끝난 뒤 버튼을 안 풀어 줌	34개 실패
오류 때 커서를 안 줌	16개 실패
새로 전송할 때 이전 오류를 안 지움	2개 실패
입력하면 오류를 안 지움	2개 실패
`preventDefault()` 제거(브라우저의 기본 전송이 일어나 페이지가 이동)	시험이 중단
연결 반환값을 항상 `true`로	1개 실패
422 문구 제거	2개 실패
연결 실패 문구 제거	2개 실패
이메일 공백을 안 뗌(`trim()` 제거)	**실패 없음**

마지막 한 가지는 시험의 빈틈이 아니라 브라우저의 성질입니다. 입력칸이 `type="email"`이면 브라우저가 앞뒤 공백을 값에서 스스로 지웁니다(`you@example.com` 을 넣으면 `value`가 `you@example.com`). 직접 확인했습니다. 그래서 `trim()`을 지워도 결과가 같습니다. `trim()`은 입력칸의 종류가 바뀌거나 값을 프로그램이 넣는 경우를 위한 방어입니다.

5. 페이지 연결

`home.js`와 `join.js`에 한 줄씩 더했습니다.

```
initRecoverForms(document.getElementById('login-modal'));
```

이제 실제 화면에서 `Send reset link`와 `Send reminder`를 누르면 서버로 요청이 갑니다. 기존 시험(열기 28개, 주소 31개, 연결 24개, 보낸 뒤 화면 36개, 함수별 시험)도 그대로 통과합니다.

6. 아직 안 된 것

`send again`은 아직 잠긴 채입니다. 다음 함수가 1분 뒤에 켜고, 누르면 `modal.recoverLast`로 다시 보냅니다.

7. 개정 (2026-10-15): 늦게 도착한 응답과 옛 요청의 잠금

173번 시험을 하다가 이 함수에도 같은 오류가 있음을 발견해 고쳤습니다.

문제	고침
요청을 보낸 뒤 응답이 오기 전에 `Back to Sign In`을 누르거나 모달을 닫으면, **늦게 온 응답이 보낸 뒤 화면을 띄우거나 타이머를 되살림**	`resetRecoverState()`가 `modal.recoverEpoch`(세대)를 올리고, 요청은 시작 때의 값을 기억해 응답이 왔을 때 달라져 있으면 성공도 실패도 **버림**
첫 요청이 안 끝난 채 화면을 떠났다가 다시 보내면 옛 요청의 `data-busy` 때문에 **새 전송이 무시됨**	처음으로 돌릴 때 `data-busy`와 버튼 잠금을 풀고, 요청의 `finally`는 **같은 세대일 때만** 풂
마지막으로 보낸 이메일이 메모리에 남음	처음으로 돌릴 때 `modal.recoverLast = null`

시험 `late.mjs`(14개)와 `resend_extra.mjs`(12개)가 이 경우들을 확인하고, 이 수정을 되돌리는 변이(M11, M13~M16, M12)를 모두 잡습니다. 위 53개 시험과 변이 시험 결과는 그대로입니다.

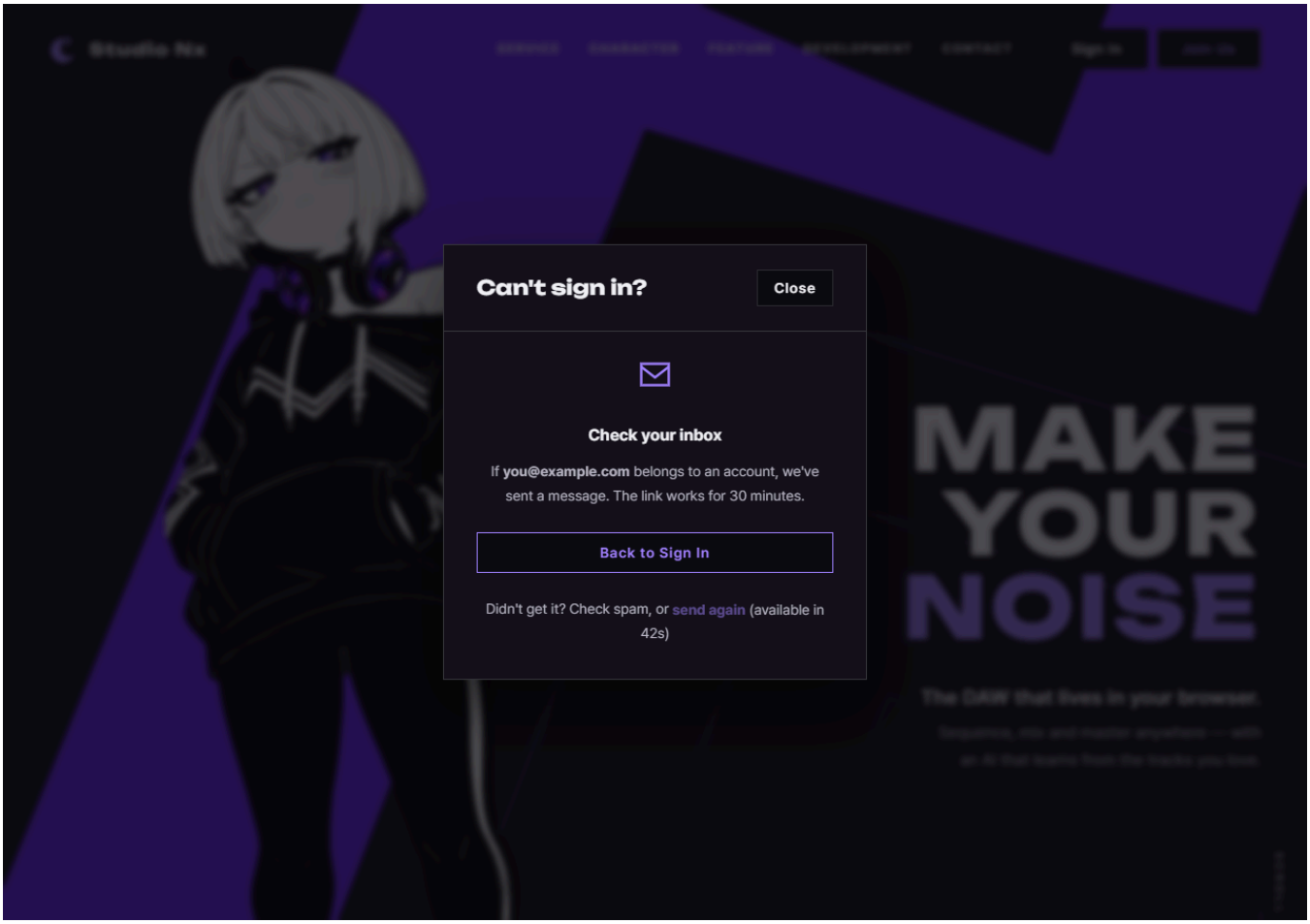
JS 172 — `startResendCooldown()`와 `stopResendCooldown()` (`send again`의 1분 기다림)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (상수 `RESEND_COOLDOWN_SECONDS = 60`)
작업	P1-28 계정 찾기 화면: JS 여덟 번째, 아홉 번째 함수 (짝으로 만듦)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	171 `initRecoverForms()`
다음	다시 보내기: `send again`을 누르면 `modal.recoverLast`로 다시 보내고, 폼 전송 성공 때 이 시간 세기를 시작하고, 모달을 닫을 때 멈춤

1. 하는 일

보낸 뒤 화면의 `send again` 버튼을 **1분 동안 잠그고 남은 시간을 보여 주다가** 0이 되면 풀어 줍니다.

함수	하는 일
`startResendCooldown(modal, seconds)`	버튼을 잠그고 옆에 `(available in 58s)`를 1초마다 갱신합니다. 0이 되면 글자를 지우고 버튼을 풀고 타이머를 정리합니다. `seconds`를 생략하면 60초입니다
`stopResendCooldown(modal)`	타이머를 끊고 **처음 상태**(잠김, 안내 글자 없음)로 되돌립니다. 화면을 떠나거나 모달을 닫을 때, 그리고 새로 시작하기 전에 부릅니다



입력	결과
`start(modal)`	60초
`start(modal, 2.2)`	`(available in 3s)`(올림)
세는 중에 `start`를 다시 부름	처음부터 다시 셈
`seconds`가 `0`, `-1`, `NaN`, `Infinity`, 문자열, `null`, `true`, 배열, 객체	`false`이고 아무것도 안 바뀜
버튼이 없음	`false`
안내 글자 칸이 없음	오류 없이 동작

2. 코드

```
export const RESEND_COOLDOWN_SECONDS = 60;

export function stopResendCooldown(modal) {
  if (!modal) return false;
  if (modal.recoverTimer) {
    window.clearInterval(modal.recoverTimer);
    modal.recoverTimer = null;
  }
  const button = modal.querySelector('#recover-resend');
  const note = modal.querySelector('#recover-resend-wait');
  if (note) note.textContent = '';
  if (!button) return false;
  button.disabled = true;
  return true;
}

export function startResendCooldown(modal, seconds) {
  const total = seconds === undefined ? RESEND_COOLDOWN_SECONDS : seconds;
  if (!modal || typeof total !== 'number' || !isFinite(total) || total <= 0) return false;
  const button = modal.querySelector('#recover-resend');
  const note = modal.querySelector('#recover-resend-wait');
  if (!button) return false;

  stopResendCooldown(modal);
  const endAt = Date.now() + total * 1000;

  const render = function () {
    const left = Math.ceil((endAt - Date.now()) / 1000);
    if (left <= 0) {
      stopResendCooldown(modal);
      button.disabled = false;   // stop 은 잠그므로 끝난 뒤에 풀음
      return;
    }
    if (note) note.textContent = '(available in ' + left + 's)';
  };

  button.disabled = true;
  render();
  modal.recoverTimer = window.setInterval(render, 1000);
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const stopResendCooldown = (modal) => {
  if (!modal) return false;
  clearInterval(modal.recoverTimer);
  modal.recoverTimer = null;
  const button = modal.querySelector('#recover-resend');
  const note = modal.querySelector('#recover-resend-wait');
  if (note) note.textContent = '';
  if (!button) return false;
  button.disabled = true;
  return true;
};

export const startResendCooldown = (modal, seconds = RESEND_COOLDOWN_SECONDS) => {
  if (!modal || !Number.isFinite(seconds) || seconds <= 0) return false;
  const button = modal.querySelector('#recover-resend');
  const note = modal.querySelector('#recover-resend-wait');
  if (!button) return false;

  stopResendCooldown(modal);
  const endAt = Date.now() + seconds * 1000;
  const render = () => {
    const left = Math.ceil((endAt - Date.now()) / 1000);
    if (left <= 0) { stopResendCooldown(modal); button.disabled = false; return; }
    if (note) note.textContent = `(available in ${left}s)`;
  };
  render();
  modal.recoverTimer = setInterval(render, 1000);
  return true;
};
```

최신 문법	실제 코드	뜻
`seconds = 기본값`	`seconds === undefined ? … : seconds`	생략하면 기본값을 씁니다
`Number.isFinite(seconds)`	`typeof … === 'number' && isFinite(…)`	숫자이면서 유한한지 한 번에 봅니다. `Number.isFinite`는 문자열을 자동 변환하지 않아서 `'5'`를 거부합니다
템플릿 문자열 `` `…${left}…` ``	`'…' + left + '…'`	글자와 값을 잇습니다
`clearInterval(undefined)`	`if (modal.recoverTimer)` 먼저 확인	타이머가 없어도 오류가 나지 않아 확인이 필요 없습니다

문법 풀이

식	뜻
`window.setInterval(render, 1000)`	`render`를 1초마다 부릅니다. 반환된 번호로 `clearInterval`이 끊습니다
`Date.now()`	지금 시각(밀리초)
`Math.ceil(…)`	올림: 2.2초 남았으면 3으로 보여 줍니다
`modal.recoverTimer`	타이머 번호를 모달 요소에 붙여 두어, 나중에(다른 함수가) 끊을 수 있게 합니다

3. 왜 이렇게

선택	이유
남은 시간을 끝나는 시각에서 지금 시각을 빼서 계산함	`setInterval`의 횟수를 세면(`60, 59, 58…`) 탭이 백그라운드에 있을 때 브라우저가 타이머를 느리게 돌려서 시간이 어긋납니다. 끝나는 시각을 기준으로 하면 몇 초를 놓쳐도 다음 틱에 바로잡힙니다. 시험에서 시계를 2분 앞당기자 다음 틱에 풀렸습니다
올림(`ceil`)	0.4초 남았는데 `0s`로 보이면서 아직 잠겨 있는 어색한 순간이 생깁니다
`stop`이 버튼을 **잠그는** 쪽으로	모달을 닫을 때(`stop`) 다음에 열면 `send again`은 새로 보낼 때까지 잠겨 있어야 합니다. 보낸 뒤 화면이 없는 동안 풀려 있으면 안 됩니다
끝난 뒤의 해제만 `start`가 함	`stop`이 잠그므로 시간이 끝나는 곳에서만 `disabled = false`를 둡니다
다시 시작하면 이전 타이머를 끊음	타이머가 둘 돌면 글자가 번갈아 깜빡이고 한쪽이 먼저 버튼을 풉니다
잘못된 시간은 `false`(`'5'`도)	문자열 `'5'`를 `Date.now() + '5' * 1000`으로 우연히 계산할 수는 있지만, 부르는 쪽의 실수를 조용히 넘기지 않습니다
타이머 번호를 `modal.recoverTimer`에 둠	전역 변수 대신 모달에 붙여서 여러 모달이 생겨도 섞이지 않고, 모달을 닫는 쪽이 이 번호로 끊을 수 있습니다
1분(60초)	사용자와 정한 값입니다. 서버의 한도(이메일 3번, 10분)보다 짧아서, 누르는 대로 다 통과하면 3번 만에 막힙니다. 그래서 다음 함수는 429가 오면 그 오류를 보여 주어야 합니다

4. 시험 (실제 서버 페이지, 실제 시간, 54개 통과)

`home.html`, `join.html`에서 같은 시험을 했습니다. 짧은 시간(1~5초)으로 실제로 기다리며 확인했습니다.

확인	결과
기본 시간 상수가 60, 시작 전에는 잠겨 있고 글자가 없음	✓ ✓ ✓ ✓
시작하면 `true`, 잠김, `(available in 3s)`, 타이머 있음	✓ ✓
1초쯤 뒤 2초로 줄어듦	✓ ✓
끝나면 풀리고 글자가 지워지고 타이머가 정리됨, 그 뒤 타이머 누수 없음	✓ (4개)
세는 중에 다시 시작하면 처음부터, 타이머가 하나뿐	✓ (4개)
`stop`: 잠기고 글자가 지워지고 타이머 정리, 시간이 지나도 풀리지 않음, 또 불러도 오류 없음	✓ (6개)
풀린 뒤에 `stop`하면 다시 잠김	✓ ✓
시계가 앞서가면 다음 틱에 풀림	✓ ✓
시간을 생략하면 60초, 2.2초는 3으로 올림	✓ ✓ ✓ ✓
잘못된 시간 10종은 `false`이고 안 바뀜	✓ (20개)
모달이 `null`이면 둘 다 `false`	✓ ✓
버튼이 없으면 `false`, 안내 글자 칸이 없어도 오류 없음	✓ ✓

시험을 확인하는 시험. 일부러 틀리게 바꿔 시험이 잡는지 보았습니다. 백그라운드로 12가지를 돌렸고, 끝난 뒤 파일이 원본과 같은지 확인했습니다.

바꾼 것	결과
끝난 뒤 버튼을 안 풀어 줌	8개 실패

바꾼 것	결과
끝나도 타이머를 안 정리	4개 실패
다시 시작할 때 이전 타이머를 안 끊음	2개 실패
횟수로 셈(시계를 무시)	8개 실패
올림 대신 버림	8개 실패
`stop`이 버튼을 안 잠금	2개 실패
`stop`이 글자를 안 지움	10개 실패
`stop`이 타이머를 안 끊음	2개 실패
시간 검사 제거	20개 실패
기본값 제거	2개 실패
글자 형식 변경	12개 실패
시작할 때 `button.disabled = true`를 안 함	**실패 없음**

마지막은 시험의 빈틈이 아니라 **중복**입니다. `start`가 바로 앞에서 `stopResendCooldown()`을 부르는데, 그 함수가 이미 버튼을 잠그기 때문에 `start`의 `button.disabled = true`가 같은 일을 한 번 더 합니다. 지워도 동작이 같습니다. 읽는 사람이 "시작하면 잠긴다"를 바로 알 수 있도록 남겨 둡니다.

5. 아직 이 함수를 부르는 곳이 없습니다

다음 함수가 폼 전송이 성공했을 때 `startResendCooldown()`을, 모달을 닫거나 화면을 떠날 때 `stopResendCooldown()`을 부릅니다. 그 전에는 `send again`이 잠긴 채입니다.

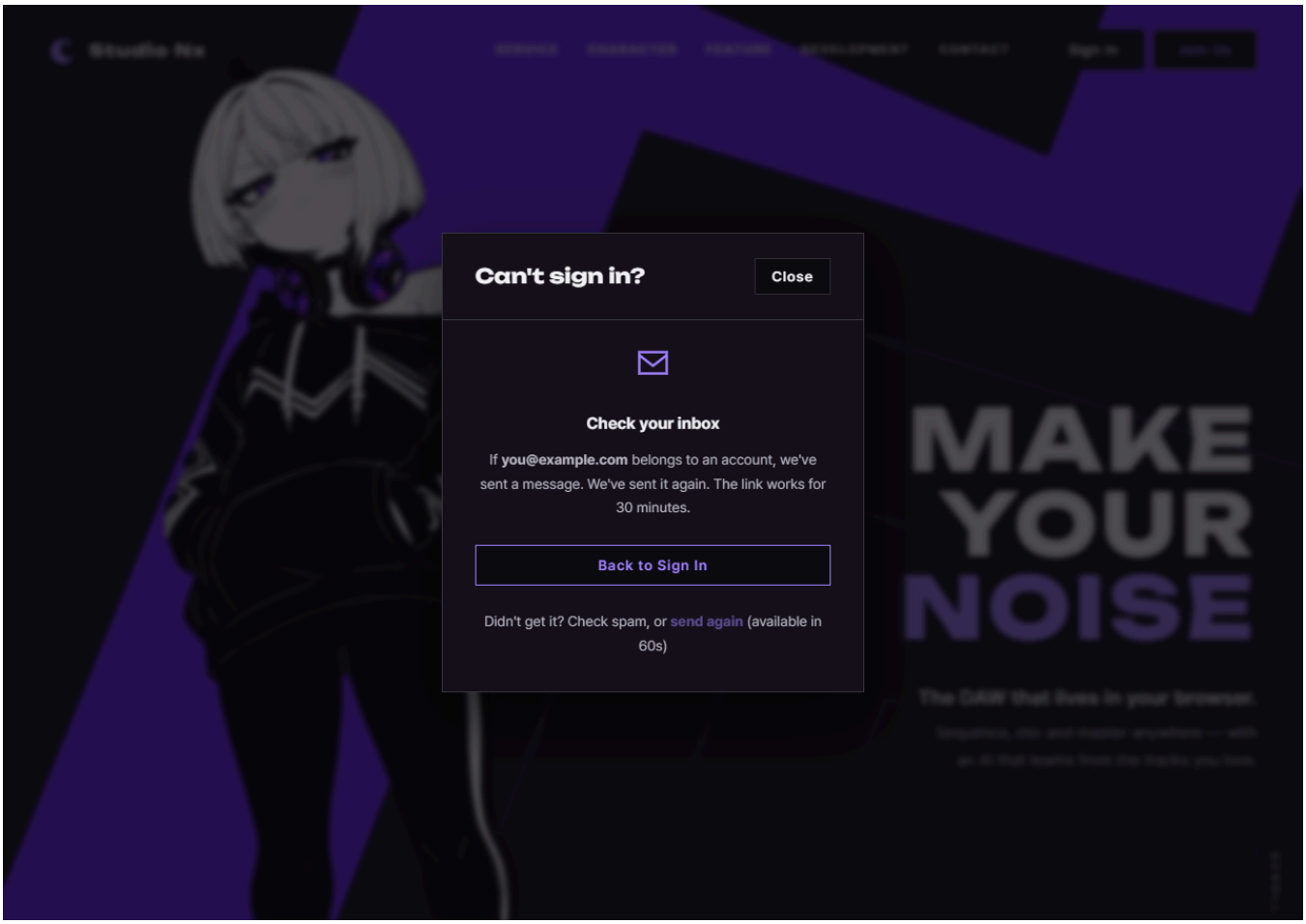
JS 173 — `initRecoverResend()` (`send again`으로 다시 보내기)

항목	내용
파일	`studionx/frontend/static/js/core/recover.js` (보낸 뒤 화면에 `#recover-resend-error` 칸을 `login-modal.html`에 더함)
작업	P1-28 계정 찾기 화면: JS 열 번째 함수. 이 함수를 만들면서 앞 함수(`initRecoverForms`, `resetRecoverState`)의 오류 두 가지도 함께 고침
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	172 `startResendCooldown()`, `stopResendCooldown()`
다음	`resetPassword.html` (HTML, CSS, JS)

1. 하는 일

보낸 뒤 화면의 `send again`을 누르면 **마지막으로 성공한 요청을 같은 종류, 같은 이메일로 다시 보냅니다.**

결과	화면
성공	보충 문장이 "We've sent it again. ..."로 바뀌고 새 1분을 셈
429	폼 전송과 같은 문구("N분 뒤에"), 서버가 알려 준 `retry_after`(초)만큼 **다시 잠금**
그 밖의 실패(500, 연결 실패)	문구를 보이고 버튼을 풀어 다시 누를 수 있게 함
보낸 적이 없음(`modal.recoverLast`가 없음)	아무것도 하지 않음



보내는 동안 버튼을 잠그고, 같은 버튼의 두 번째 클릭은 무시합니다(`data-busy`).

2. 코드

```
export function initRecoverResend(modal) {
  if (!modal) return false;
  const button = modal.querySelector('#recover-resend');
  if (!button) return false;
  const errorBox = modal.querySelector('#recover-resend-error');

  button.onclick = async function () {
    const last = modal.recoverLast;
    if (!last || button.getAttribute('data-busy') === 'true') return;
    let spec = null;
    for (const candidate of RECOVER_FORMS) if (candidate.kind === last.kind) spec = candidate;
    if (spec === null) return;

    button.setAttribute('data-busy', 'true');
    button.disabled = true;
    const epoch = modal.recoverEpoch || 0;
    if (errorBox) { errorBox.textContent = ''; errorBox.hidden = true; }
    try {
      await api.post(spec.path, { email: last.email });
      if ((modal.recoverEpoch || 0) !== epoch) return;   // 기다리는 사이 화면을 떠남: 늦은 응답은 버림
      const detail = modal.querySelector('#recover-sent-detail');
      if (detail) detail.textContent = "We've sent it again. " + SENT_DETAIL[last.kind];
      startResendCooldown(modal);
    } catch (err) {
      if ((modal.recoverEpoch || 0) !== epoch) return;
      if (errorBox) { errorBox.textContent = recoverErrorText(err); errorBox.hidden = false; }
      const wait = err && err.status === 429 && err.details && typeof err.details.retry_after === 'numbe
r'
        ? err.details.retry_after : 0;
      if (wait > 0) startResendCooldown(modal, wait);
      else button.disabled = false;
    } finally {
      if ((modal.recoverEpoch || 0) === epoch) button.removeAttribute('data-busy');
    }
  };
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initRecoverResend = (modal) => {
  const button = modal?.querySelector('#recover-resend');
  if (!button) return false;
  const errorBox = modal.querySelector('#recover-resend-error');

  button.addEventListener('click', async () => {
    const last = modal.recoverLast;
    const spec = RECOVER_FORMS.find((f) => f.kind === last?.kind);
    if (!spec || button.dataset.busy) return;
```

```
    button.dataset.busy = 'true';
    button.disabled = true;
    const epoch = modal.recoverEpoch ?? 0;
    errorBox && Object.assign(errorBox, { textContent: '', hidden: true });
    try {
      await api.post(spec.path, { email: last.email });
      if ((modal.recoverEpoch ?? 0) !== epoch) return;
      modal.querySelector('#recover-sent-detail').textContent = `We've sent it again. ${SENT_DETAIL[las
t.kind]}`;
      startResendCooldown(modal);
    } catch (err) {
      if ((modal.recoverEpoch ?? 0) !== epoch) return;
      if (errorBox) Object.assign(errorBox, { textContent: recoverErrorText(err), hidden: false });
      const wait = err?.status === 429 ? (err.details?.retry_after ?? 0) : 0;
      wait > 0 ? startResendCooldown(modal, wait) : (button.disabled = false);
    } finally {
      if ((modal.recoverEpoch ?? 0) === epoch) delete button.dataset.busy;
    }
  });
  return true;
};
```

최신 문법	실제 코드	뜻
`RECOVER_FORMS.find(…)`	`for`로 돌며 변수에 담기	종류가 같은 폼을 찾습니다
`last?.kind`	`if (!last …)` 먼저 확인	값이 없으면 건너뜁니다
`modal.recoverEpoch ?? 0`	`modal.recoverEpoch || 0`	값이 없으면 0으로 봅니다
`err.details?.retry_after ?? 0`	`err.details && typeof … === 'number' ? … : 0`	값이 없거나 숫자가 아니면 0
템플릿 문자열	문자열 이음(`+`)	글자와 값을 잇습니다
`button.dataset.busy`	`getAttribute('data-busy')`	`data-` 속성

문법 풀이

식	뜻
`button.onclick = async function …`	처리기를 대입합니다. 두 번 불러도 겹치지 않습니다
`recoverEpoch`	화면을 처음으로 돌릴 때마다 올라가는 "세대" 번호입니다. 요청이 시작될 때의 번호를 기억해 두었다가, 응답이 왔을 때 달라져 있으면 그 사이 화면을 떠난 것이라 응답을 버립니다
`finally`의 `=== epoch` 조건	옛 요청이 끝나면서 **그 사이 시작된 새 요청의 잠금까지** 풀지 않게 합니다
`retry_after`	서버가 알려 준 "몇 초 뒤에 다시 할 수 있는지"

3. 왜 이렇게

선택	이유
마지막 **성공한** 요청만 다시 보냄 (`recoverLast`)	입력칸에 새로 적은 값이 아니라 사용자가 이미 보낸 이메일을 다시 보내야 합니다. 보낸 뒤 화면에는 입력칸이 없습니다
429면 **서버가 준 시간만큼** 다시 잠금	서버의 한도(이메일 3번, 10분)는 화면의 1분보다 길어서, 1분마다 눌러도 3번째부터 막힙니다. 서버가 풀기 전에 또 누르게 하지 않습니다

선택	이유
그 밖의 실패는 버튼을 **풀어 줌**	일시적인 오류(연결 끊김)로 막아 두면 사용자가 포기합니다
서버 문구를 그대로 보여 주지 않음	`recoverErrorText()`(171번)를 함께 씁니다. 내부 사정이 새거나 가입 여부를 짐작하게 하는 문구를 막습니다
성공하면 새 1분	연속으로 눌러 메일을 퍼붓는 일을 막습니다

4. 시험하며 발견한 오류 두 가지 (이 함수와 171번 모두에 있었음)

4-1. 늦게 도착한 응답이 닫힌 화면을 바꿈

요청을 보낸 뒤 **응답이 오기 전에** `Back to Sign In`을 누르거나 모달을 닫으면, 나중에 온 응답이 다음 일을 했습니다.

- 닫힌 모달에서 보낸 뒤 화면을 띄움(다시 열면 로그인 화면이 아니라 보낸 뒤 화면이 보임)
- 멈춰 둔 타이머를 되살림
- 이미 처리된 새 요청의 결과를 덮어씀

고침: `resetRecoverState()`가 호출될 때마다 `modal.recoverEpoch`를 올리고, 요청은 시작할 때의 값을 기억해 응답이 오면 비교해서 **다르면 버립니다.** `initRecoverForms()`(171)와 이 함수 모두 같은 방식입니다.

4-2. 옛 요청이 새 전송을 막음

첫 요청이 아직 안 끝났는데 화면을 떠났다가 다시 보내면, 옛 요청이 달아 둔 "보내는 중" 표시(`data-busy`) 때문에 **새 전송이 무시**됐습니다.

고침: `resetRecoverState()`가 `data-busy`와 버튼 잠금을 풀고, 옛 요청이 끝날 때의 `finally`는 **같은 세대일 때만** 풉니다(그 사이 새 요청이 시작되었다면 건드리지 않음).

4-3. 개인정보를 오래 남기지 않음

화면을 처음으로 돌릴 때 `modal.recoverLast`(마지막으로 보낸 이메일)도 `null`로 지웁니다.

5. 시험 (실제 서버 페이지, `fetch`를 가짜로 바꿈)

시험	개수	내용
`resend.mjs`	37	처음 보낸 직후 잠김(60초), 풀린 뒤 클릭하면 같은 주소, 같은 이메일, 한 번 보냄, 성공 문장과 새 1분, 429(서버 시간으로 다시 잠금, 시간 모름은 풀림), 500과 연결 실패는 문구와 버튼 풀림, 오류 뒤 성공하면 오류가 지워짐, 연속 클릭은 한 번, `Back`하면 타이머가 멈춤, Email 탭은 다른 주소, 모달을 닫으면 타이머 정리, 다시 열면 60초부터, `recoverLast` 없으면 무시
`late.mjs`	14	(A) 응답 전에 Back, (B) 응답 전에 모달 닫기, (C) 늦은 실패 응답, (D) 가만히 기다리면 평소대로, (E) `send again`의 늦은 응답, (F) 먼저 보낸 요청의 늦은 응답이 두 번째 요청을 덮어쓰지 않음
`resend_extra.mjs`	12	남긴 값이 없거나 알 수 없는 종류면 요청도 오류도 없음, 보내는 중 표시가 있으면 무시, `Back`과 모달 닫기로 `recoverLast`가 지워짐, 옛 요청이 끝나도 새 요청의 잠금은 그대로

기존 시험(열기 28, 주소 31, 연결 24, 보낸 뒤 화면 36, 폼 53, 시간 세기 54, 탭 40과 39, 화면 전환 27, 로그인 링크 6과 처리방침 링크 17 등)도 모두 그대로 통과합니다.

시험을 확인하는 시험 (변이 시험 18가지)

소스를 일부러 틀리게 바꿔 시험이 잡는지 확인했습니다. 백그라운드로 돌렸고, 끝난 뒤 파일이 원본과 같은지 확인했습니다.

#	바꾼 것	처음 결과	보강 뒤
M1	`recoverLast`가 없는데도 진행	**못 잡음**	4개 실패
M2	보내는 중 검사(`data-busy`) 제거	**못 잡음**	2개 실패
M3	성공 때 새 1분을 안 셈	4개 실패	
M4	"다시 보냈다" 문장 제거	4개 실패	
M5	429에서 서버 시간으로 안 잠금	2개 실패	
M6	실패해도 버튼을 안 풀어 줌	8개 실패	
M7	서버 문구를 그대로 보여 줌	8개 실패	
M8	새로 누를 때 이전 오류를 안 지움	2개 실패	
M9	종류별 주소를 무시(항상 재설정으로)	2개 실패	
M10	화면을 처음으로 돌릴 때 타이머를 안 멈춤	6개 실패	
M11	처음으로 돌릴 때 세대를 안 올림	10개 실패	
M12	처음으로 돌릴 때 `recoverLast`를 안 지움	**못 잡음**	4개 실패
M13	처음으로 돌릴 때 보내는 중 표시를 안 풂	6개 실패	
M14	폼의 늦은 **성공** 응답을 안 버림	6개 실패	
M15	폼의 늦은 **실패** 응답을 안 버림	2개 실패	
M16	폼의 `finally`가 항상 잠금을 풂	**못 잡음**	2개 실패
M17	다시 보내기의 늦은 응답을 안 버림	2개 실패	
M18	버튼이 없는데도 진행	시험이 예외로 중단	

처음에 못 잡은 네 가지는 모두 시험의 빈틈이었고 보강했습니다. 이전 함수에서 "중복 방어"라고 정리했던 것과 달리 이번에는 서로 다른 일을 막는 검사였습니다.

- **M1:** 남긴 값이 없으면 요청은 가지 않지만 **처리되지 않은 예외**가 났습니다. 요청 개수만 세던 시험은 이를 몰랐습니다. 이제 자바스크립트 예외 개수까지 확인합니다.
- **M2:** 처음에는 "버튼이 `disabled`라서 클릭이 닿지 않으니 중복 방어"라고 판단했는데, 이것은 클릭만 본 것이었습니다. `data-busy`가 붙었는데 잠금이 풀려 있는 순간(예: 오류 직후)은 `disabled`가 막지 못합니다. 이 상태를 만들어 시험했습니다.
- **M12:** `recoverLast`가 지워지는지를 아무도 확인하지 않았습니다. 개인정보를 남기는 문제라서 가장 중요한 빈틈이었습니다.
- **M16:** 옛 요청의 `finally`가 새 요청의 잠금을 풀어 버리는 경쟁을 만들려면 옛 요청이 끝나는 순간에 새 요청이 아직 가는 중이어야 합니다. 응답 시간을 다르게 한 가짜 서버로 이 순간을 만들었습니다.

6. 페이지 연결

`home.js`와 `join.js`에 한 줄씩 더했습니다.

```
initRecoverResend(document.getElementById('login-modal'));
```

폼 전송이 성공하면 `startResendCooldown()`이 불리고, 화면을 처음으로 돌릴 때 `stopResendCooldown()`이 불립니다.

JS 174 — `readResetToken()` (주소의 `#token=` 읽고 곧바로 지우기)

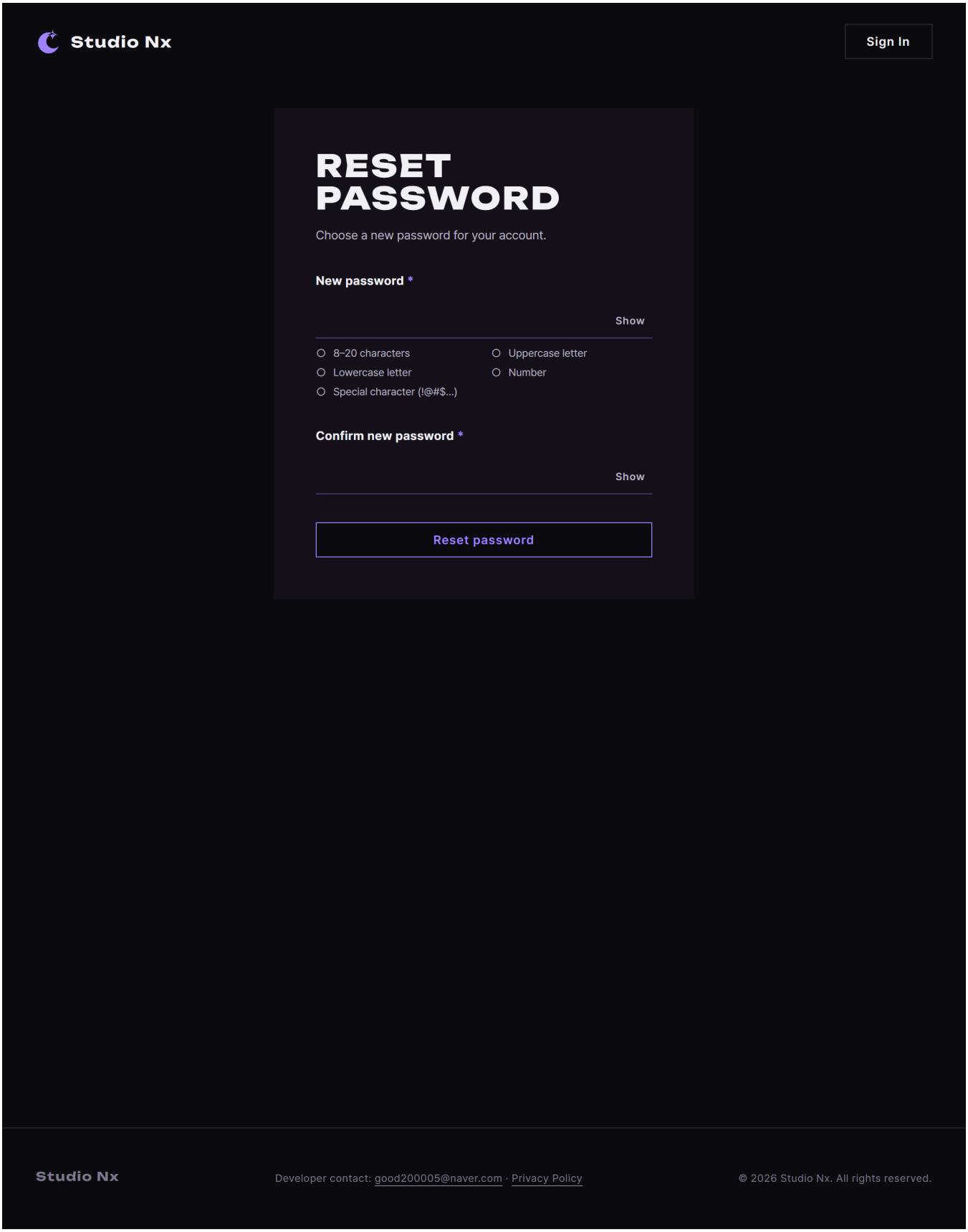
항목	내용
파일	`studionx/frontend/static/js/pages/resetPassword.js` (새 파일)
작업	P1-28 비밀번호 재설정 화면: JS 첫 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	개요 70 §5 (HTML과 CSS), JS 173
다음	175 `initResetPasswordRules()`, 그다음 `initResetForm()`

1. 하는 일

재설정 메일의 링크로 들어오면 주소가 `resetPassword.html#token=<원문>` 입니다. 이 함수는 그 `#token=` 뒤의 값을 읽어 **이 파일 안에만** 두고, 주소창에서는 **곧바로 지웁니다.**

주소	돌려주는 값	주소창
`#token=abc-123_XYZ`	`abc-123_XYZ`	해시를 지웁니다
`#token=` (빈 값)	null	해시를 지웁니다
해시 없음	null	그대로 둡니다
`#something` (다른 해시)	null	그대로 둡니다
`?x=1#token=tok`	`tok`	쿼리는 남기고 해시만 지웁니다

주소창은 이렇게 정리합니다: `location.pathname + location.search`, 즉 경로와 쿼리만 남깁니다.



2. 코드

```
/** 재설정 토큰 (이 파일 안에서만 씀). 주소창에서 지운 뒤 여기에만 두고 localStorage 에는 두지 않는다 */
let resetToken = null;
```

```
export function readResetToken() {
  const hash = location.hash || '';
  const prefix = '#token=';
  let token = null;

  if (hash.indexOf(prefix) === 0) {
    const value = hash.slice(prefix.length);
    if (value !== '') token = value;
    // 주소창에서 지움 (경로와 쿼리만 남김)
    history.replaceState(null, '', location.pathname + location.search);
  }

  resetToken = token;
  return resetToken;
}
```

최신 문법으로 쓰면 (공부용)

```
export const readResetToken = () => {
  const hash = location.hash ?? '';
  const PREFIX = '#token=';
  const token = hash.startsWith(PREFIX) ? hash.slice(PREFIX.length) || null : null;
  if (hash.startsWith(PREFIX)) history.replaceState(null, '', location.pathname + location.search);
  resetToken = token;
  return token;
};
```

최신 문법	실제 코드	뜻
`hash.startsWith(PREFIX)`	`hash.indexOf(PREFIX) === 0`	해시가 `#token=` 로 시작하는지 봅니다
`hash.slice(PREFIX.length) || null`	`if (value !== '') token = value;`	빈 값이면 null 로 봅니다
(삼항 연산자)	`if` 문	조건에 따라 값을 고릅니다

문법 풀이

식	뜻
`location.hash`	주소에서 `#` 뒤 부분입니다. `#token=abc` 이면 `#token=abc` 입니다
`hash.indexOf(prefix) === 0`	문자열이 `prefix` 로 시작하면 0 입니다. 시작하지 않으면 -1 이나 다른 값입니다
`hash.slice(prefix.length)`	`#token=` 만큼 잘라 낸 나머지, 즉 토큰 값입니다
`history.replaceState(null, '', 주소)`	새 기록을 남기지 않고 주소만 바꿉니다(뒤로 가기 기록이 늘지 않음)

3. 왜 이렇게

선택	이유
읽은 값을 **이 파일 안 변수에만** 둠	토큰은 비밀번호를 바꿀 수 있는 열쇠입니다. `localStorage`나 쿠키에 두면 다른 스크립트가 읽거나 기록에 남을 수 있습니다
주소창에서 **곧바로** 지움	해시는 서버로 가지 않지만, 새로 고침이나 뒤로 가기, 주소 복사로 토큰이 남을 수 있습니다. 지우면 남지 않습니다
`history.replaceState` 로 지움	`location.hash = ''` 는 뒤로 가기 기록을 남깁니다. `replaceState` 는 남기지 않습니다

선택	이유
`<meta name="referrer" content="no-referrer">` 와 함께 씀	이 페이지에서 다른 곳으로 갈 때 주소(Referer)를 넘기지 않아 링크가 새지 않습니다
`#token=` 로 **시작할 때만** 지움	다른 해시(`#something`)는 이 페이지의 것이 아니므로 건드리지 않습니다
빈 값(`#token=`)은 토큰 없음으로 봄	토큰은 `secrets.token_urlsafe(32)` 라 값이 반드시 있습니다. 빈 값이면 잘못된 링크입니다
토큰이 없으면 null 을 돌려줌	부르는 쪽(`initResetForm`)이 "무효 링크" 화면을 보여 줄 수 있습니다

4. 시험 (실제 서버, 헤드리스 Chrome)

`docs/handoff/p1-28_tests/reset_token.mjs` 로 시험합니다. 실제 페이지를 열고 주소를 바꿔 가며 값을 확인합니다.

시험	결과
`#token=abc-123_XYZ` 를 그대로 읽음	통과
토큰을 읽은 뒤 주소창에서 해시를 지움	통과
`#token=` (빈 값)은 토큰 없음, 해시는 지움	통과
해시가 없으면 토큰 없음	통과
다른 해시(`#something`)는 토큰 없음이고 지우지 않음	통과
`?x=1#token=tok` 은 쿼리를 남기고 해시만 지움	통과

여섯 묶음, 여덟 확인이 모두 통과했습니다.

> 페이지가 완성된 뒤에는(`initResetForm` 추가) 페이지가 뜰 때 이 함수가 토큰을 먼저 읽어 소비합니다. 그래서 시험은 해시를 한 번 더 주소에 넣고(`history.replaceState`) 함수를 직접 불러 확인합니다.

5. 페이지 연결

이 함수는 아직 페이지 하단에서 부르지 않습니다. 다음 함수인 `initResetForm` 이 폼을 준비할 때 가장 먼저 불러, 토큰이 없으면 "무효 링크" 화면을 보여 주게 합니다.

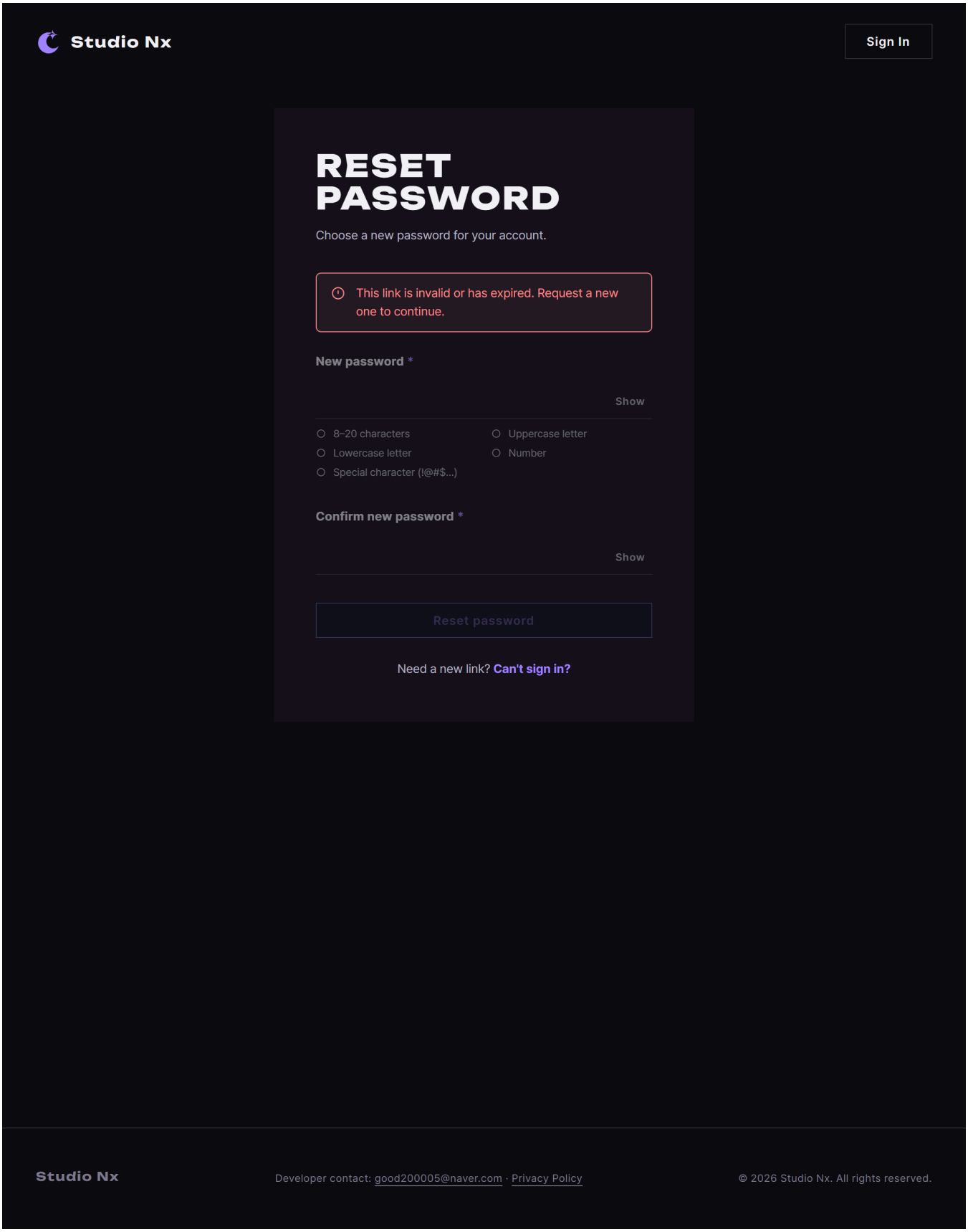
JS 175 — `initResetPasswordRules()` (규칙 표시와 확인 칸 일치)

항목	내용
파일	`studionx/frontend/static/js/pages/resetPassword.js`
작업	P1-28 비밀번호 재설정 화면: JS 두 번째 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	174 `readResetToken()`
다음	`initResetForm()` (전송, 성공, 무효 링크, 규칙 오류, 연결 실패)

1. 하는 일

새 비밀번호 칸에 입력할 때마다 규칙 다섯 줄의 표시를 바꾸고, 확인 칸이 다르면 안내합니다. 가입 화면(`join.js`의 `initPasswordRules`)과 **같은 규칙, 같은 문구**를 씁니다(`core/password.js`를 함께 씁니다).

때	화면
규칙을 채운 줄	회색 ○ 가 초록 ✓ 로 바뀜(`is-met`)
허용되지 않는 문자(한글, 공백 등)	`Use only English letters, numbers and symbols.`
규칙이 하나라도 안 맞은 채 칸을 벗어남	`Check the password requirements.`
확인 칸이 다름(비밀번호만큼 다 쳤거나 칸을 벗어남)	`Passwords don't match.`
확인 칸이 맞거나 비어 있음	안내를 지움



2. 코드

```
export function initResetPasswordRules() {
  const password = document.getElementById('reset-password');
  const confirm = document.getElementById('reset-password2');
```

```
  const passwordField = document.getElementById('reset-password-field');
  const confirmField = document.getElementById('reset-password2-field');
  const items = document.querySelectorAll('#reset-password-rules li[data-rule]');
  if (!password || !confirm || !passwordField || !confirmField) return false;

  const allMet = function () {
    for (const name of Object.keys(PASSWORD_RULES)) {
      if (!PASSWORD_RULES[name](password.value)) return false;
    }
    return true;
  };

  const showRules = function () {
    for (const item of items) {
      const rule = PASSWORD_RULES[item.dataset.rule];
      const met = rule ? rule(password.value) : false;
      item.classList.toggle('is-met', met);
      const state = item.querySelector('[data-rule-state]');
      if (state) state.textContent = met ? ', met' : ', not met';
    }
  };

  const checkConfirm = function (force) {
    if (confirm.value === '') {
      setFieldState(confirmField, null);
    } else if (confirm.value === password.value) {
      setFieldState(confirmField, null);
    } else if (force || confirm.value.length >= password.value.length) {
      setFieldState(confirmField, 'error', "Passwords don't match.");
    }
  };

  password.oninput = function () {
    showRules();
    if (!PASSWORD_ALLOWED.test(password.value)) {
      setFieldState(passwordField, 'error', 'Use only English letters, numbers and symbols.');
    } else {
      setFieldState(passwordField, null);
    }
    if (confirm.value !== '') checkConfirm(true);
  };
  password.onblur = function () {
    if (password.value !== '' && PASSWORD_ALLOWED.test(password.value) && !allMet()) {
      setFieldState(passwordField, 'error', 'Check the password requirements.');
    }
  };
  confirm.oninput = function () { checkConfirm(false); };
  confirm.onblur = function () { checkConfirm(true); };
  showRules();
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initResetPasswordRules = () => {
  const q = (id) => document.getElementById(id);
```

```
  const password = q('reset-password'), confirm = q('reset-password2');
  const passwordField = q('reset-password-field'), confirmField = q('reset-password2-field');
  const items = document.querySelectorAll('#reset-password-rules li[data-rule]');
  if (!password || !confirm || !passwordField || !confirmField) return false;

  const allMet = () => Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
  const showRules = () => items.forEach((item) => {
    const met = PASSWORD_RULES[item.dataset.rule]?.(password.value) ?? false;
    item.classList.toggle('is-met', met);
    const state = item.querySelector('[data-rule-state]');
    if (state) state.textContent = met ? ', met' : ', not met';
  });
  const checkConfirm = (force = false) => {
    const bad = confirm.value !== '' && confirm.value !== password.value;
    if (!bad || force || confirm.value.length >= password.value.length) {
      setFieldState(confirmField, bad ? 'error' : null, bad ? "Passwords don't match." : '');
    }
  };

  password.oninput = () => {
    showRules();
    setFieldState(passwordField, PASSWORD_ALLOWED.test(password.value) ? null : 'error',
      PASSWORD_ALLOWED.test(password.value) ? '' : 'Use only English letters, numbers and symbols.');
    if (confirm.value !== '') checkConfirm(true);
  };
  password.onblur = () => {
    const ok = password.value !== '' && PASSWORD_ALLOWED.test(password.value) && !allMet();
    if (ok) setFieldState(passwordField, 'error', 'Check the password requirements.');
  };
  confirm.oninput = () => checkConfirm(false);
  confirm.onblur = () => checkConfirm(true);
  showRules();
  return true;
};
```

최신 문법	실제 코드	뜻
`Object.values(...).every(...)`	`for` 로 돌며 `return false`	하나라도 안 맞으면 실패입니다
`rule?.(값) ?? false`	`rule ? rule(값) : false`	규칙이 없으면 실패로 봅니다
`items.forEach(...)`	`for (const item of items)`	줄마다 같은 일을 합니다
`force = false` (기본값)	`if (force || ...)`	값을 안 넘기면 false 로 봅니다

문법 풀이

식	뜻
`password.oninput = function () { … }`	처리기를 **대입**합니다. `addEventListener` 와 달리 두 번 불러도 겹치지 않습니다
`item.dataset.rule`	`data-rule="length"` 의 `length` 값입니다(`data-` 속성)
`classList.toggle('is-met', met)`	`met` 이 참이면 붙이고 거짓이면 뺍니다
`[data-rule-state]`	화면 낭독기용 글자(`sr-only`)입니다. 색으로만 알리는 규칙을 소리로도 알려 줍니다
`PASSWORD_ALLOWED.test(값)`	값이 허용 문자(영문, 숫자, 기호)로만 되어 있는지 봅니다

식	뜻
`checkConfirm(true)` (blur)	칸을 벗어나면 다 쳤는지와 상관없이 불일치를 알립니다

3. 왜 이렇게

선택	이유
가입 화면과 **같은** `core/password.js`	규칙과 문구가 화면마다 달라지면 사용자가 헷갈립니다. 한 곳에서만 고치게 합니다
허용 문자 오류와 규칙 미달 오류를 **나눔**	한글을 쳤을 때는 "규칙을 확인하세요"보다 "영문, 숫자, 기호만 쓰세요"가 맞습니다
확인 칸은 **입력 중에는 길이 이상일 때만**, 벗어나면 항상	아직 다 치지도 않았는데 불일치라고 하면 성급합니다. 다 친 뒤에 알립니다
처리기를 **대입**(`oninput`, `onblur`)	두 번 불러도 앞 처리기를 덮어써 겹치지 않습니다
화면 낭독기용 글자(`data-rule-state`)도 갱신	색(○/✓)만으로는 알 수 없는 사용자를 위해 소리로도 알립니다
칸이 없으면 false	다른 페이지에 이 파일이 잘못 붙어도 오류가 나지 않습니다

4. 시험 (실제 서버, 헤드리스 Chrome)

`docs/handoff/p1-28_tests/reset_rules.mjs` 로 시험합니다. 실제 키 입력까지 넣습니다.

시험	결과
실제 타이핑 `Passw0rd!` 로 규칙 다섯 줄 모두 `is-met`	통과
`abc` 는 length 와 lower 만 met	통과
규칙 미달로 칸을 벗어나면 `Check the password requirements.`	통과
허용되지 않는 문자(한글)에 `Use only English letters, numbers and symbols.`	통과
허용 문자로 돌아오면 오류 문구가 사라짐	통과
확인 칸이 비밀번호보다 짧을 때는 입력 중 안내하지 않음	통과
확인 칸을 벗어나면 `Passwords don't match.`	통과
확인 칸이 길이 이상이고 다르면 입력 중 알림	통과
확인 칸이 맞으면 오류 없음	통과
두 번 불러도 true	통과
화면 낭독기용 글자(`data-rule-state`)가 갱신	통과

열두 확인이 모두 통과했습니다.

> 페이지가 완성된 뒤에는(`initResetForm` 추가) 토큰 없이 열면 폼이 잠기므로(입력·전송 막음), 시험은 `#token=` 을 붙여 엽니다.

시험을 고친 것

처음 시험에서 두 가지가 **시험의 잘못**이었습니다.

- 확인 칸 불일치를 시험할 때 비밀번호보다 짧은 값을 넣어 두고 안내를 기대했습니다. 하지만 입력 중에는 길이 이상일 때만 알리므로 안내가 없었습니다. 값과 기대를 규칙에 맞게 고쳤습니다.
- 칸을 벗어나는 시험에서 `blur()` 만 불렀습니다. 포커스가 없던 칸은 `blur` 이벤트가 나지 않아, **포커스를 준 뒤** 벗어나도록 고쳤습니다.

5. 페이지 연결

`resetPassword.js` 아래쪽에서 부릅니다.

```
await includePartials();
initModalFallback();
initAuthState();
initFormAlerts();
initPasswordToggles();
initResetPasswordRules();
```

`#reset-password` 칸이 없으면 아무것도 연결하지 않고 false 를 돌려줍니다.

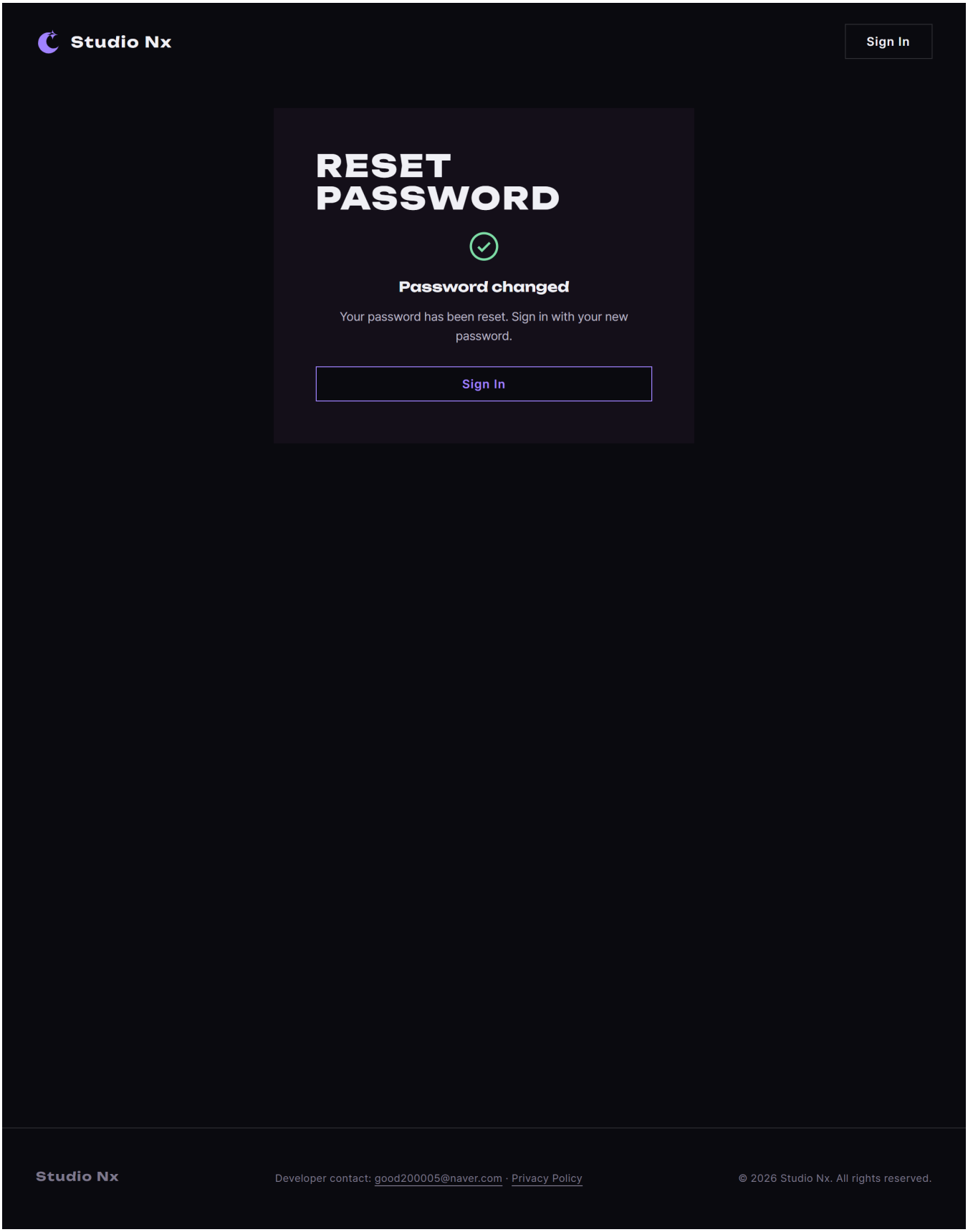
JS 176 — `initResetForm()` (재설정 전송, 무효 링크, 성공)

항목	내용
파일	`studionx/frontend/static/js/pages/resetPassword.js`
작업	P1-28 비밀번호 재설정 화면: JS 마지막(세 번째) 함수
작성	Claude (AI), 2026-10-15, 사용자 검토
이전	175 `initResetPasswordRules()`, 개요 70 §5
다음	없음 (이 함수로 `resetPassword.html`의 JS가 끝남)

1. 하는 일

페이지를 열면 토큰을 확인하고, 폼을 보낸 결과에 따라 화면 세 가지 중 하나를 보여 줍니다.

때	화면
토큰 없음	위쪽에 무효 경고, 폼은 흐리게 잠김, 아래에 `Can't sign in?`(다시 요청)
보내기 성공(204)	초록 체크, `Password changed`, `Sign In` (자동 로그인은 하지 않음)
400 (링크 틀림·만료·이미 씀)	무효 경고와 잠긴 폼, 다시 요청 줄
422 (규칙 위반)	비밀번호 칸에 `Check the password requirements.` (링크는 그대로 씀)
연결 실패, 그 밖(500 등)	폼 아래에 문구, 버튼을 풀어 다시 시도



전송 전에 화면에서 규칙과 확인을 **먼저** 검사합니다. 틀린 칸이 있으면 그 칸에 문구를 넣고 커서를 옮긴 뒤 서버로 보내지 않습니다.

2. 코드

```
export function initResetForm() {
  const form = document.getElementById('reset-form');
  const card = document.querySelector('.reset__card');
  const invalid = document.getElementById('reset-invalid');
  const invalidText = document.getElementById('reset-invalid-text');
  const recoverRow = document.getElementById('reset-recover-row');
  const done = document.getElementById('reset-done');
  const sub = document.getElementById('reset-sub');
  const password = document.getElementById('reset-password');
  const confirm = document.getElementById('reset-password2');
  const passwordField = document.getElementById('reset-password-field');
  const confirmField = document.getElementById('reset-password2-field');
  const error = document.getElementById('reset-error');
  if (!form || !password || !confirm) return false;
  const submit = form.querySelector('button[type="submit"]');

  const isLocked = function () {
    return card ? card.classList.contains('is-locked') : false;
  };

  const showInvalid = function () {
    if (invalid) invalid.hidden = false;
    if (card) card.classList.add('is-locked');
    for (const control of form.querySelectorAll('input, button')) control.disabled = true;
    if (recoverRow) recoverRow.hidden = false;
  };

  const showDone = function () {
    form.hidden = true;
    if (sub) sub.hidden = true;
    if (invalid) invalid.hidden = true;
    if (recoverRow) recoverRow.hidden = true;
    if (done) done.hidden = false;
  };

  const token = resetToken || readResetToken();
  if (!token) {
    showInvalid();
    return true;
  }

  form.onsubmit = async function (event) {
    event.preventDefault();
    if (form.getAttribute('data-busy') === 'true') return;
    if (error) error.hidden = true;

    let problem = null;
    if (!PASSWORD_ALLOWED.test(password.value)) {
      setFieldState(passwordField, 'error', 'Use only English letters, numbers and symbols.');
      problem = password;
    } else {
      let met = true;
      for (const name of Object.keys(PASSWORD_RULES)) {
        if (!PASSWORD_RULES[name](password.value)) met = false;
```

```
      }
      if (!met) {
        setFieldState(passwordField, 'error', 'Check the password requirements.');
        problem = password;
      }
    }
    if (!problem && confirm.value !== password.value) {
      setFieldState(confirmField, 'error', "Passwords don't match.");
      problem = confirm;
    }
    if (problem) {
      problem.focus();
      return;
    }

    form.setAttribute('data-busy', 'true');
    if (submit) submit.disabled = true;
    try {
      await api.post(
        '/auth/password-reset/confirm',
        { token: token, new_password: password.value },
        { auth: false },
      );
      showDone();
    } catch (err) {
      if (err && err.status === 400) {
        showInvalid();
      } else if (err && err.status === 422) {
        setFieldState(passwordField, 'error', 'Check the password requirements.');
        password.focus();
      } else if (error) {
        error.textContent = err && err.status === 0
          ? "Can't reach the server. Check your connection and try again."
          : 'Something went wrong. Please try again.';
        error.hidden = false;
      }
    } finally {
      form.removeAttribute('data-busy');
      if (submit && !isLocked()) submit.disabled = false;
    }
  };
  return true;
}
```

최신 문법으로 쓰면 (공부용)

```
export const initResetForm = () => {
  const form = document.getElementById('reset-form');
  const card = document.querySelector('.reset__card');
  const invalid = document.getElementById('reset-invalid');
  const recoverRow = document.getElementById('reset-recover-row');
  const done = document.getElementById('reset-done');
  const password = document.getElementById('reset-password'), confirm = document.getElementById('reset-p
assword2');
  if (!form || !password || !confirm) return false;
  const submit = form.querySelector('button[type="submit"]');
```

```
  const showInvalid = () => {
    invalid && (invalid.hidden = false);
    card?.classList.add('is-locked');
    form.querySelectorAll('input, button').forEach((el) => (el.disabled = true));
    recoverRow && (recoverRow.hidden = false);
  };
  const showDone = () => [form, document.getElementById('reset-sub'), invalid, recoverRow]
    .forEach((el) => el && (el.hidden = true)),
    done && (done.hidden = false);

  const token = resetToken ?? readResetToken();
  if (!token) { showInvalid(); return true; }

  form.onsubmit = async (event) => {
    event.preventDefault();
    if (form.dataset.busy) return;
    const message = !PASSWORD_ALLOWED.test(password.value) ? 'Use only English letters, numbers and symb
ols.'
      : Object.values(PASSWORD_RULES).every((r) => r(password.value)) ? '' : 'Check the password require
ments.';
    if (message) return setFieldState(document.getElementById('reset-password-field'), 'error', messag
e);
    if (confirm.value !== password.value) return setFieldState(document.getElementById('reset-password2-
field'), 'error', "Passwords don't match.");

    form.dataset.busy = 'true';
    try {
      await api.post('/auth/password-reset/confirm', { token, new_password: password.value }, { auth: fa
lse });
      showDone();
    } catch (err) {
      err?.status === 400 ? showInvalid()
        : err?.status === 422 ? setFieldState(document.getElementById('reset-password-field'), 'error',
'Check the password requirements.')
        : document.getElementById('reset-error') && Object.assign(document.getElementById('reset-erro
r'), { hidden: false, textContent: err?.status === 0 ? "Can't reach the server. Check your connection an
d try again." : 'Something went wrong. Please try again.' });
    } finally {
      delete form.dataset.busy;
      if (!card?.classList.contains('is-locked')) submit && (submit.disabled = false);
    }
  };
  return true;
};
```

최신 문법	실제 코드	뜻
`resetToken ?? readResetToken()`	`resetToken || readResetToken()`	토큰이 없으면 읽습니다
`form.dataset.busy`	`getAttribute('data-busy')`	`data-` 속성
`Object.values(...).every(...)`	`for` 로 돌며 `met = false`	하나라도 안 맞으면 실패
`err?.status === 400`	`err && err.status === 400`	오류일 때만 상태를 봅니다
`card?.classList`	`card ? card.classList : …`	요소가 없어도 오류가 나지 않게

문법 풀이

식	뜻
`resetToken || readResetToken()`	이미 읽은 토큰이 있으면 그대로 씁니다. 두 번 불렀을 때 해시가 이미 지워져 있어도 다시 잠기지 않습니다
`form.querySelectorAll('input, button')`	폼 안의 입력칸과 버튼을 모두 골라 한꺼번에 잠급니다
`data-busy`	보내는 중임을 나타내는 표시입니다. 두 번째 전송을 막습니다
`finally` 의 `!isLocked()`	보낸 뒤 버튼을 다시 켜되, 무효 링크로 잠근 화면에서는 켜지 않습니다
`showInvalid` / `showDone`	화면 세 가지를 함수로 나눠 두어, 어느 결과에서도 같은 모양이 되게 합니다

3. 왜 이렇게

선택	이유
화면에서 **먼저** 검사	서버에 가지도 않을 요청을 보내지 않고, 어느 칸이 틀렸는지 바로 알려 줍니다
400은 무효 링크 화면	링크가 틀렸거나 만료됐거나 이미 쓴 경우입니다. 이유를 구분해 알려 주면 추측을 돕습니다(백엔드도 같은 400을 씁니다)
422는 **칸 오류**로만	링크는 그대로 쓸 수 있으므로 폼을 잠그지 않습니다. 규칙 문구를 칸 아래에 보입니다
성공해도 **자동 로그인 없음**	재설정한 뒤 새 비밀번호로 직접 로그인하게 합니다(FR-118)
서버 문구를 그대로 보여 주지 않음	내부 사정이 새지 않게 상태 코드로 문구를 고릅니다(복구 모달의 `recoverErrorText`와 같은 이유)
토큰을 `resetToken`에만 둠	`readResetToken`(174)이 `localStorage`·쿠키에 두지 않고 이 파일에만 둡니다
두 번 불러도 안전	처리기를 `onsubmit` 대입으로 달고, 이미 읽은 토큰은 다시 읽지 않습니다

4. 시험 (실제 서버, 헤드리스 Chrome, `fetch`를 가짜로)

`docs/handoff/p1-28_tests/reset_form.mjs` 로 시험합니다.

시험	결과
유효 토큰: 잠기지 않고, 경고 숨김, 폼 보임, 다시 요청 줄 숨김	통과
규칙 미달 → 비밀번호 칸 오류, 서버로 안 보냄	통과
확인 불일치 → 확인 칸 오류, 서버로 안 보냄	통과
성공(204) → 성공 화면, 폼·안내문 숨김	통과
성공 → 한 번 보냄, 주소가 `confirm`	통과
성공 → 본문에 `token` 과 `new_password`	통과
400 → 무효 경고, 폼 잠김, 다시 요청 줄 보임	통과
400 → 폼은 보이되 입력·전송 잠김	통과
422 → 비밀번호 칸 오류, 잠기지 않고 버튼 다시 켜짐	통과
연결 실패 → 폼 아래 문구, 버튼 다시 켜짐	통과
보내는 중 두 번째 전송은 무시(한 번만)	통과
토큰 없음 → 처음부터 무효 경고와 잠긴 폼	통과
두 번 불러도 true 이고 잠기지 않음	통과

열네 확인이 모두 통과했습니다.

시험을 고친 것

처음에는 주소의 **해시만** 바꿔 가며 시험했습니다. 그런데 해시만 바꾸면 브라우저가 페이지를 다시 실리지 않아(same-document 이동) 앞 시험의 화면이 그대로 남았습니다(성공 화면이 그대로). 매번 다른 쿼리(`?r=N`)를 붙여 페이지를 새로 실리도록 고쳤고, 그 뒤 모두 통과했습니다.

5. 페이지 연결

`resetPassword.js` 아래쪽에서 순서대로 부릅니다.

```
await includePartials();
initModalFallback();
initAuthState();
initFormAlerts();
initPasswordToggles();
initResetPasswordRules();
initResetForm();
```

이 함수로 `resetPassword.html`의 JS가 끝납니다. 남은 일은 실제 서버와 실제 재설정 링크로 한 흐름을 확인하는 사이트 시험입니다.